INTRODUCTION TO VLSI

Final Project Report

gsclib045 45 nm Standard Cell Library

Siba Aasi — 213780638

Library: Siba_vlsi

Raghad Khatib — 212763296

Library: raghad_final_project

Cadence Virtuoso · Spectre · PVS (DRC / LVS) · Quantus QRC

Part 1

4-Bit ALU Design — Implementation Stage

1. Introduction

This report covers the Implementation Stage of our 4-bit ALU final project. Starting from the architecture that was defined in the Planning Stage, the whole design was entered in Cadence Virtuoso using pre-built standard cells from the gsclib045 library, simulated with Spectre, laid out at the transistor level for the adder, extracted with Quantus QRC, and verified with PVS DRC and LVS.

The work reported here answers the requirements of the implementation stage:

- A detailed logical schematic of the complete ALU, built hierarchically from gsclib045 cells (Section 3).
- Functional verification of every sub-block and of the full ALU (Section 4).
- Measurement of the total delay of the critical path at 1.2 V and at 900 mV (Section 5).
- Extraction of the maximal clock frequency (minimal clock period) at both supply voltages (Section 5).
- A detailed hierarchical floor plan of the design (Section 6).
- A full custom layout of the adder, with area comparison against the theoretical standard-cell area (Section 7).
- DRC and LVS runs on the adder layout with zero errors (Section 8).

2. Design Overview

The ALU receives three signed 4-bit inputs A[3:0], B[3:0] and C[3:0] in two's-complement representation, and produces a signed 6-bit result Y[5:0]. The computation is split over two arithmetic stages separated by pipeline registers:

$$X[4:0] = A[3:0] + B[3:0] \qquad Y[5:0] = X[4:0] - C[3:0]$$

The subtraction is not implemented with a dedicated subtractor. Instead, C is sign-extended to 5 bits, inverted bit by bit, and fed to a 5-bit Kogge–Stone adder whose carry-in is tied to logic '1', which realises two's-complement subtraction:

$$Y = X - C_{ext} = X + \overline{C_{ext}} + 1$$

Because every operand and every intermediate result passes through a register, the ALU behaves as a two-stage pipeline.

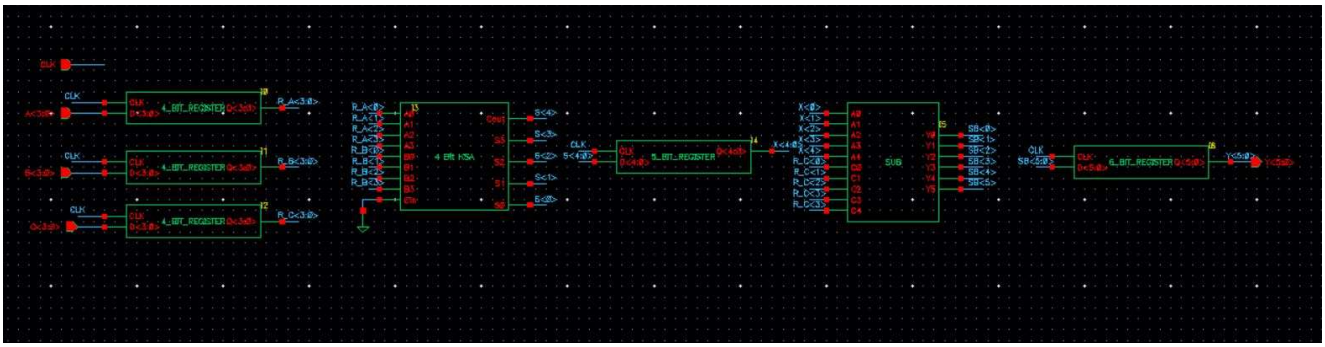


Figure 1 — Complete ALU schematic: three 4-bit input registers, the 4-bit Kogge–Stone adder, the 5-bit intermediate register X, the subtractor block and the 6-bit output register Y.

The design consists of:

- 3 × 4-bit registers (for A, B and C)
- 1 × 5-bit register (intermediate result X)
- 1 × 6-bit register (output Y)
- 1 × 4-bit Kogge–Stone adder (addition stage)
- 1 × 5-bit subtractor = 5-bit inverter + 5-bit Kogge–Stone adder with $C_{in} = 1$

3. Detailed Logical Schematic

The whole ALU was built bottom-up from gsclib045 cells. This section presents each level of the hierarchy, from the smallest leaf cells up to the complete ALU.

Registers

All registers are synchronous parallel-in / parallel-out registers built from parallel DFFHQX1 D-type flip-flops sharing a single global clock (CLK). Three widths were needed: 4 bits for the operands A, B and C, 5 bits for the intermediate sum X, and 6 bits for the final result Y.

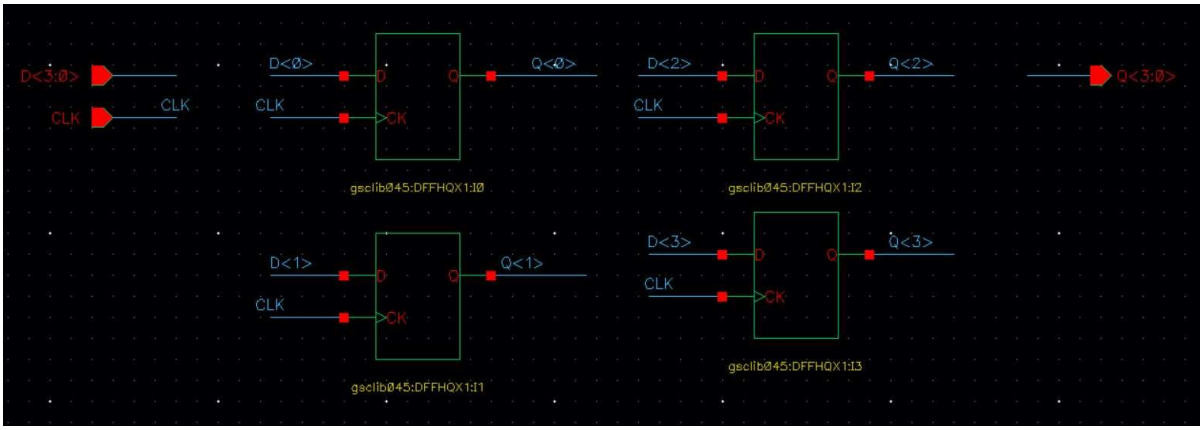


Figure 2 — 4-bit register — 4 × DFFHQX1 driven by a common CLK.

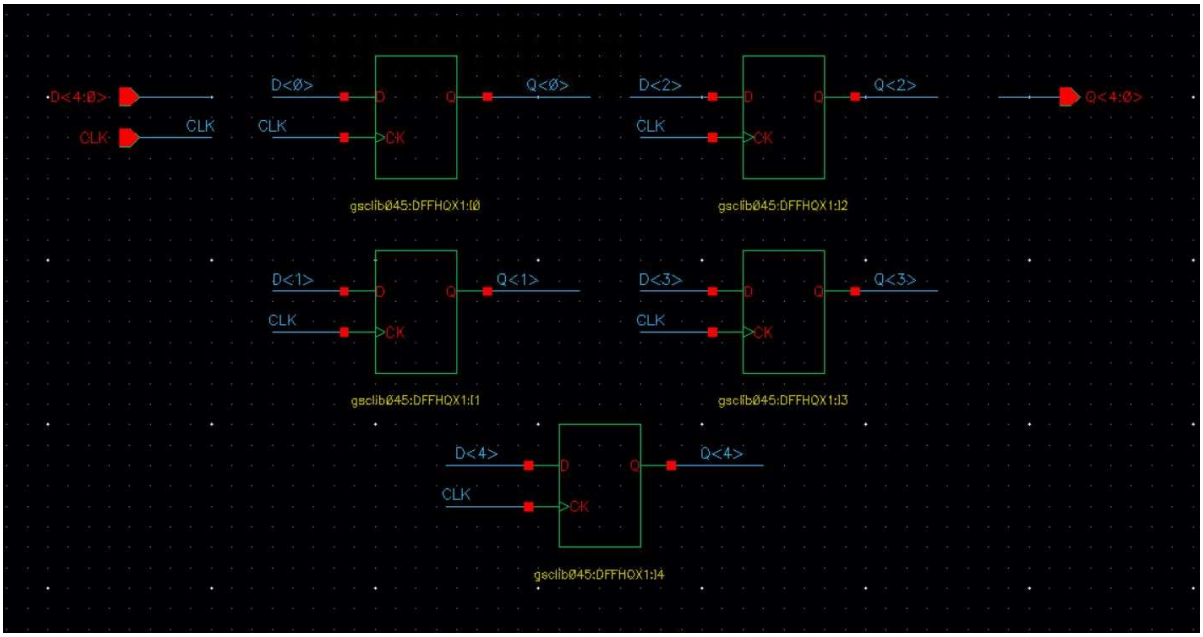


Figure 3 — 5-bit register — used for the intermediate result X[4:0].

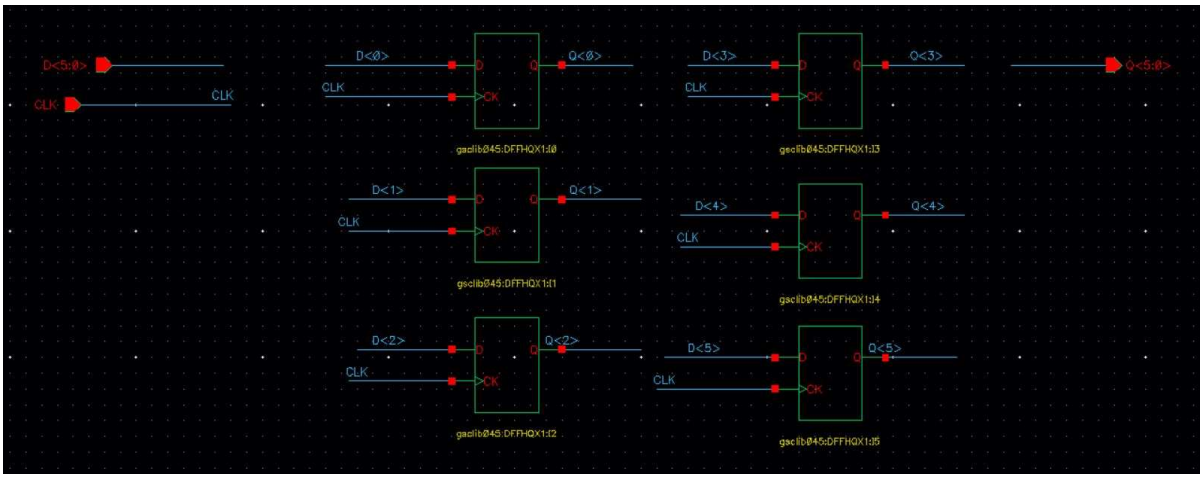


Figure 4 — 6-bit register — used for the ALU output Y[5:0].

Kogge–Stone building blocks

As explained in the planning stage, the Kogge–Stone adder is built from three layers: the PG-logic layer, the parallel-prefix network (grey and black cells), and the post-processing XOR layer.

PG logic (white cell)

The PG-logic cell is the first stage of the carry-look-ahead structure. It converts the raw operand bits into the initial propagate and generate signals:

$$P_i = A_i \oplus B_i \qquad G_i = A_i \cdot B_i$$

It is built from one XOR2X1 and one AND2X2 cell.

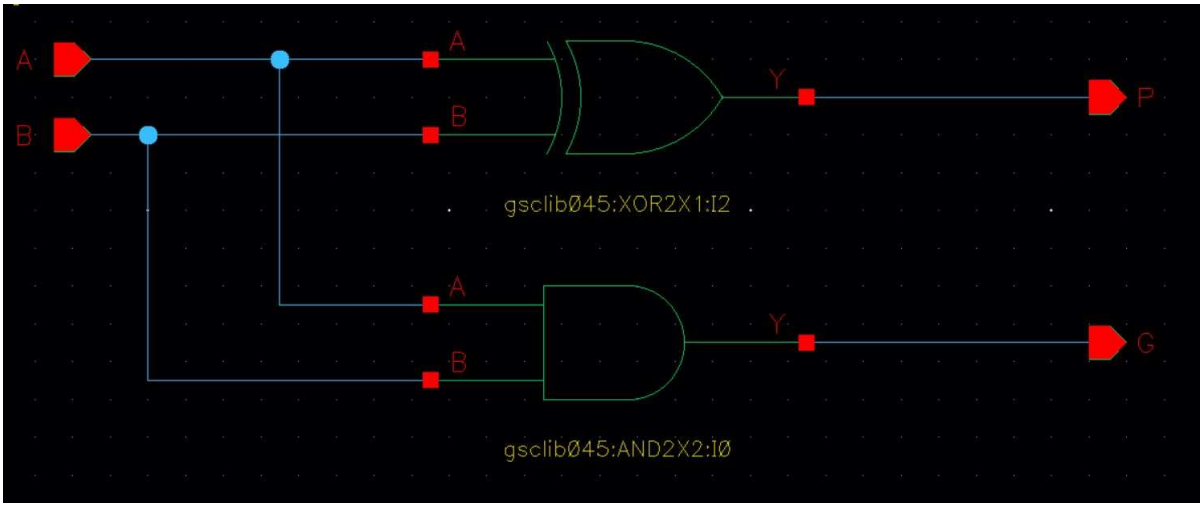


Figure 5 — PG_LOGIC cell — XOR2X1 generates the propagate signal, AND2X2 the generate signal.

Grey cell

The grey cell is the reduced version of the black cell, used at the end of a prefix branch where only the carry (generate) information is still needed. It computes only the updated generate signal, using one AND2X2 and one OR2X1:

$$G = G_i + P_i \cdot G_{i-1}$$

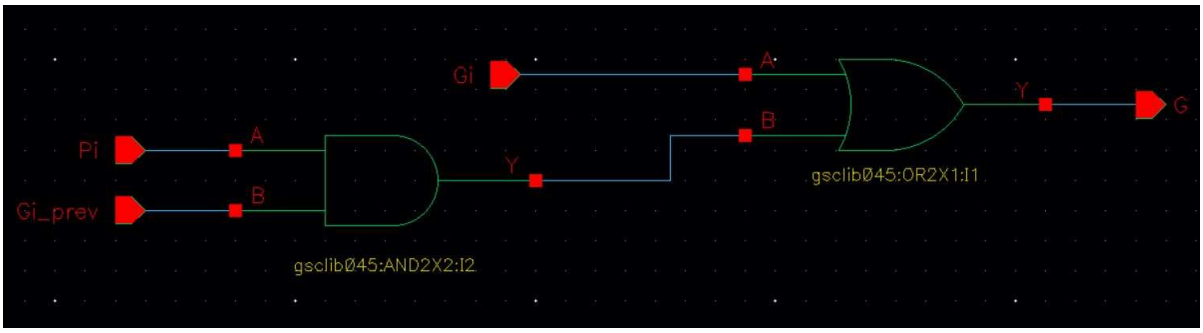


Figure 6 — GREY_CELL — computes the group generate signal only (AND2X2 + OR2X1).

Black cell

The black cell is the main building block of the prefix network. It performs the prefix operation on both the generate and the propagate signals, combining the information of the current bit with that of a lower bit. It uses two AND2X2 cells and one OR2X1:

$$G = G_i + P_i \cdot G_{i-1} \qquad P = P_i \cdot P_{i-1}$$

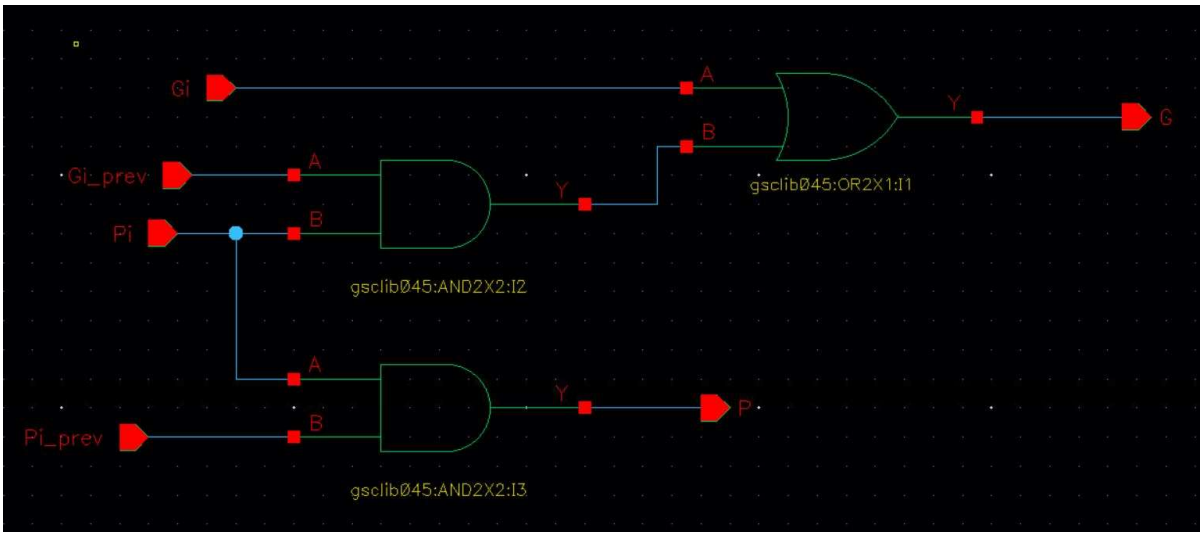


Figure 7 — BLACK_CELL — the prefix operator for both generate and propagate (2 × AND2X2 + OR2X1).

4-bit Kogge–Stone adder

The 4-bit adder is assembled from 4 PG_LOGIC cells, a two-level prefix network made of 4 black cells and 4 grey cells, and a post-processing layer of 5 XOR2X1 gates that produce S0–S3 and the carry-out. The

prefix depth is $\log_2(4) = 2$ levels, which is exactly what gives the Kogge–Stone structure its speed advantage over a ripple-carry adder.

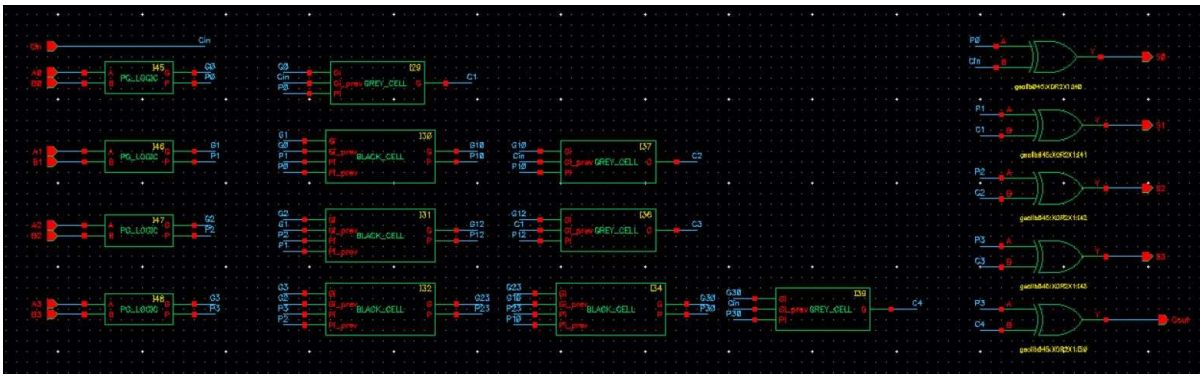


Figure 8 — 4-bit Kogge–Stone adder — PG logic (left), two-level prefix network (middle), post-processing XOR layer (right).

5-bit Kogge–Stone adder

For the subtraction stage the same structure was expanded to 5 bits: 5 PG_LOGIC cells, a three-level prefix network with 6 black cells and 5 grey cells, and 6 XOR2X1 gates producing S0–S4 and the carry-out. The extra prefix level is the reason this block is slower than the 4-bit adder, and it is what makes it the critical path of the ALU.

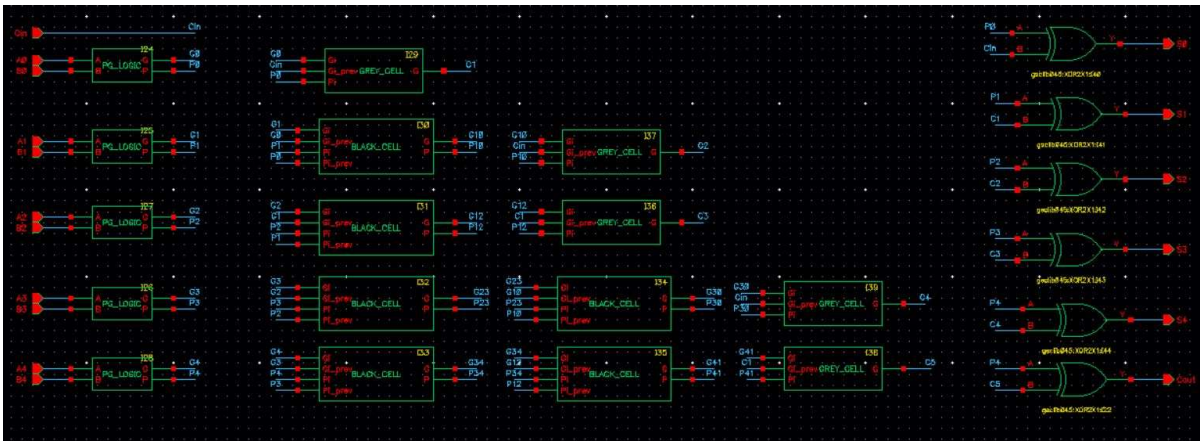


Figure 9 — 5-bit Kogge–Stone adder — expanded version of the 4-bit adder, used inside the subtractor.

Subtractor

The subtractor implements Y = X − C using two's-complement arithmetic. The registered value of C is sign-extended to 5 bits, all five bits are inverted by a 5-bit inverter block, and the result is applied to the B input of the 5-bit Kogge–Stone adder whose carry-in is tied to VDD (logic '1'). The 5-bit sum together with the carry-out forms the 6-bit output Y[5:0].

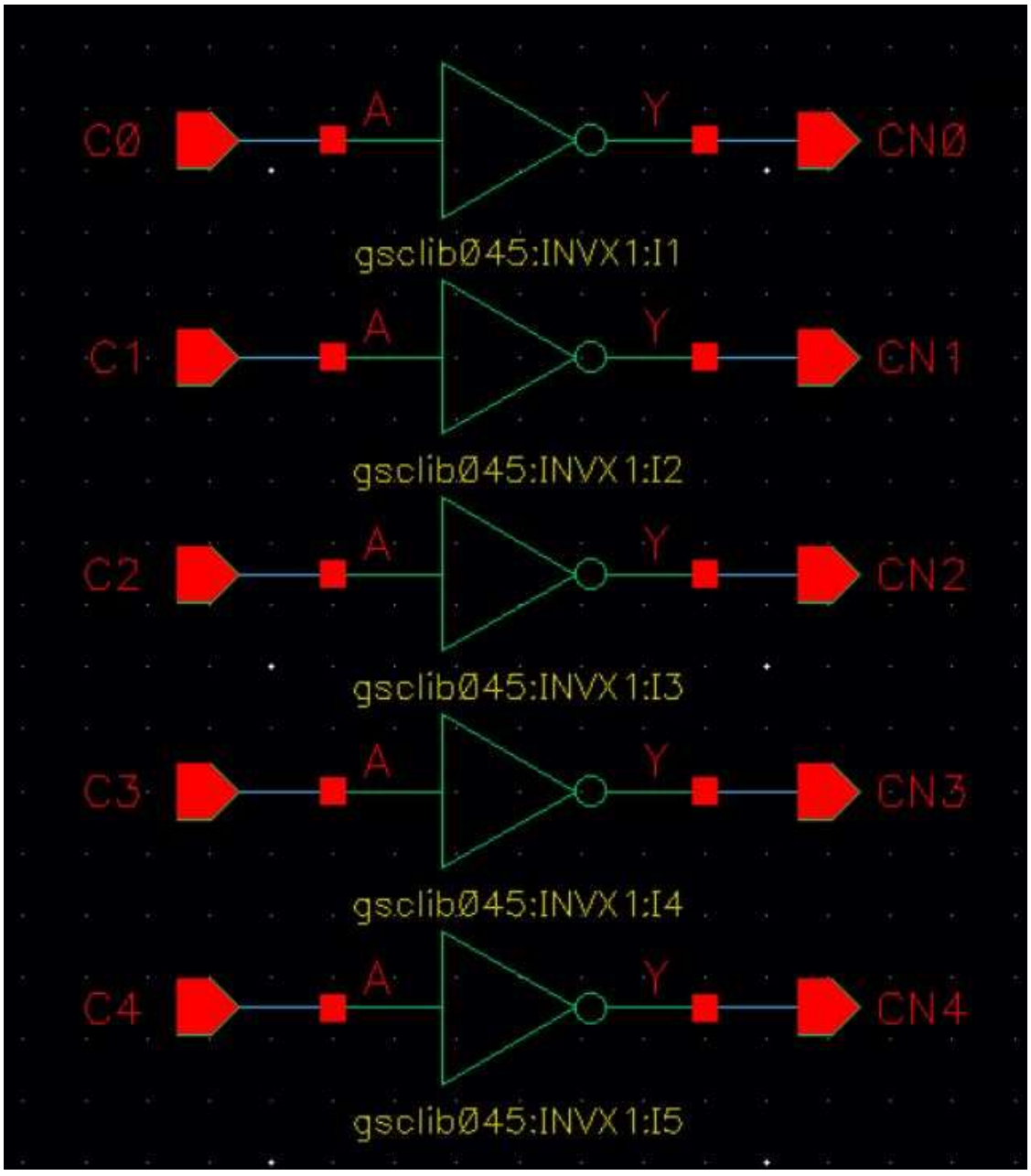


Figure 10 — 5-bit inverter block — 5 × INVX1, used to produce the one's complement of the sign-extended C.

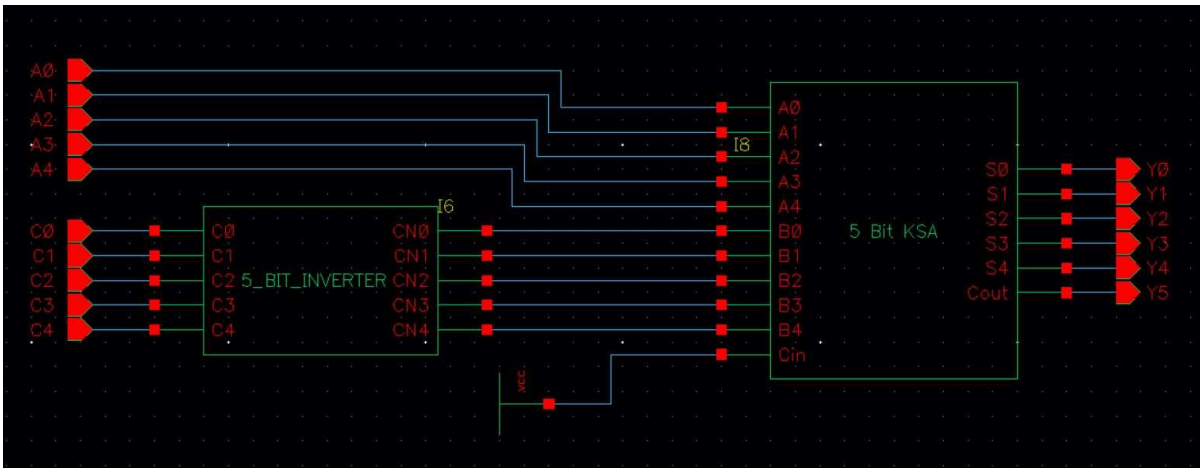


Figure 11 — Subtractor block — 5-bit inverter feeding the 5-bit Kogge–Stone adder with C_{in} = VDD.

The total theoretical standard-cell area of the ALU is approximately 329.46 μm^2.

4. Functional Verification

Every block was verified separately before being integrated, and the complete ALU was verified at the end. In all three test benches the operands are driven by independent pulse sources, VDD and VSS come from DC sources, and each output is loaded with a 10 fF capacitor so that the circuit always drives a realistic fan-out load.

4-bit Kogge–Stone adder

The adder test bench applies independent pulse sources to A[3:0], B[3:0] and C_{in}. Because the carry-in is a real input of this block, it is toggled at twice the rate of the operands, so that every operand pair is tested once with C_{in} = 0 and once with C_{in} = 1.

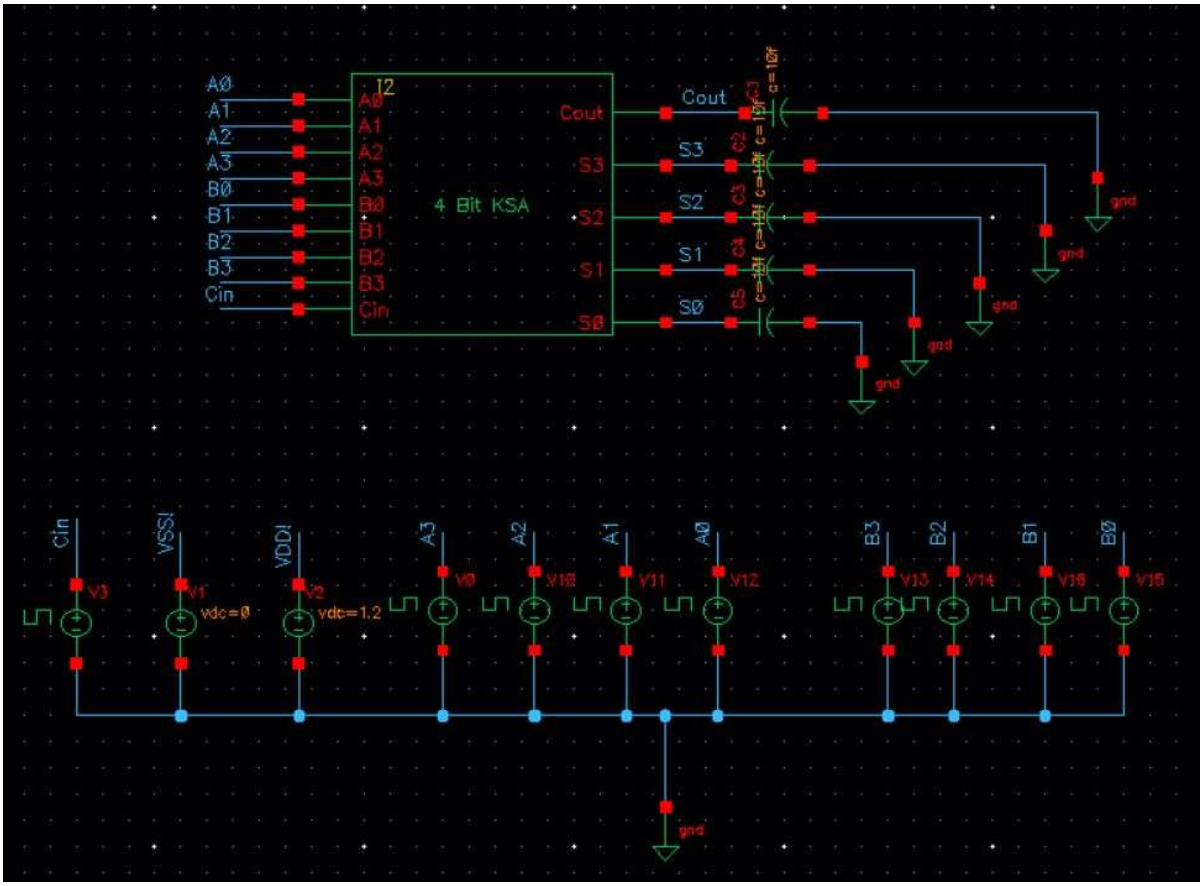


Figure 12 — Test bench of the 4-bit Kogge–Stone adder — pulse sources on the inputs and 10 fF load capacitors on S0–S3 and Cout.

The transient simulation below shows C_{in}, the two operands A and B, and the sum S decoded as signed decimal values. The operands change every 10 ns while C_{in} toggles every 5 ns, so each operand pair produces two consecutive results that differ by exactly one.



Figure 13 — Transient simulation of the 4-bit Kogge–Stone adder at VDD = 1.2 V. C_{in}, A, B and S are shown decoded as signed decimal values.

Reading the waveform vector by vector gives the following results — twenty consecutive operand / carry-in combinations, all of them correct:

C_{in}	A	B	Expected A + B + C_{in}	Measured S	Result
1	3	2	6	6	Correct
0	3	2	5	5	Correct
1	−3	2	0	0	Correct
0	−3	2	−1	−1	Correct
1	4	−2	3	3	Correct
0	4	−2	2	2	Correct
1	−4	−3	−6	−6	Correct
0	−4	−3	−7	−7	Correct
1	7	−5	3	3	Correct
0	7	−5	2	2	Correct
1	−8	7	0	0	Correct
0	−8	7	−1	−1	Correct
1	1	−1	1	1	Correct
0	1	−1	0	0	Correct
1	−6	5	0	0	Correct
0	−6	5	−1	−1	Correct
1	6	−7	0	0	Correct
0	6	−7	−1	−1	Correct
1	−2	−4	−5	−5	Correct
0	−2	−4	−6	−6	Correct

Table 1 — Vectors read from the 4-bit adder simulation. The operands cover both signs and both carry-in values; every sum matches the expected two's-complement result.

The vectors deliberately include the extreme operands of the signed 4-bit range (A = −8 and A = 7), mixed-sign pairs such as 6 + (−7), and pairs where the carry has to propagate across all four bit positions — for example −8 + 7 + 1 = 0, i.e. 1000 + 0111 + 1, which forces the carry to travel from bit 0 all the way to the carry-out through the whole prefix network. That is the worst case for the Kogge–Stone structure, and it is the same kind of vector used to measure the delays in Section 5.3.

5-bit subtractor

The subtractor was verified next. Its test bench drives the two 5-bit operands A[4:0] and C[4:0] with independent pulse sources and loads the six output bits Y[5:0] with 10 fF each. There is no carry-in source, because inside the block the carry-in of the 5-bit Kogge–Stone adder is tied permanently to VDD, as required by the two's-complement subtraction.

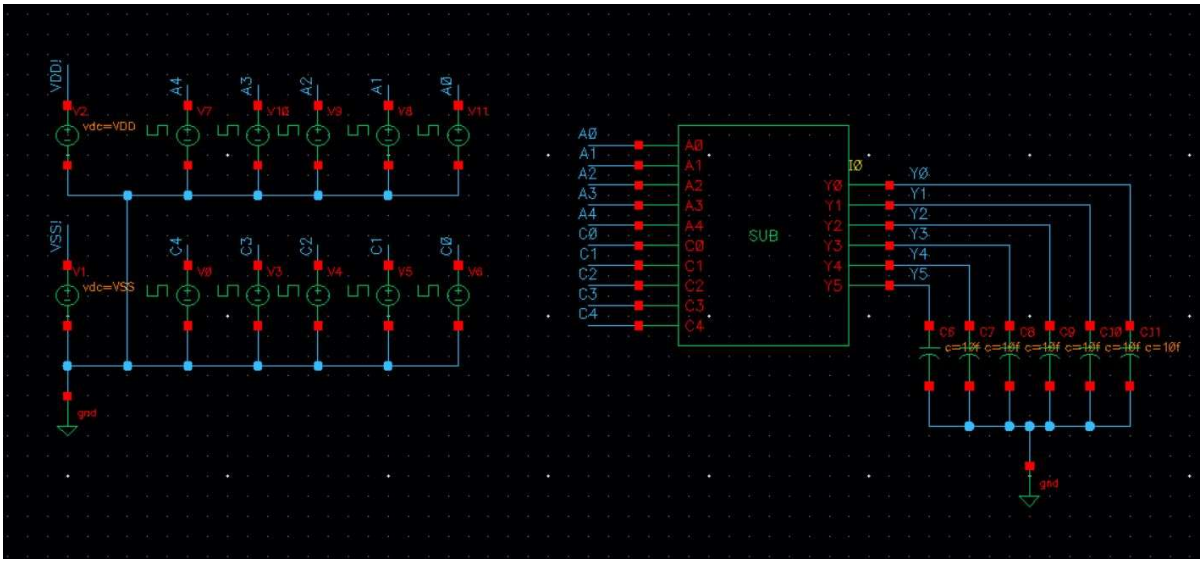


Figure 14 — Test bench of the 5-bit subtractor — pulse sources on A[4:0] and C[4:0], and 10 fF load capacitors on Y0–Y5.

The simulation confirms the relation Y = A − C over ten operand pairs spanning the whole signed 5-bit input range (−16 … +15):

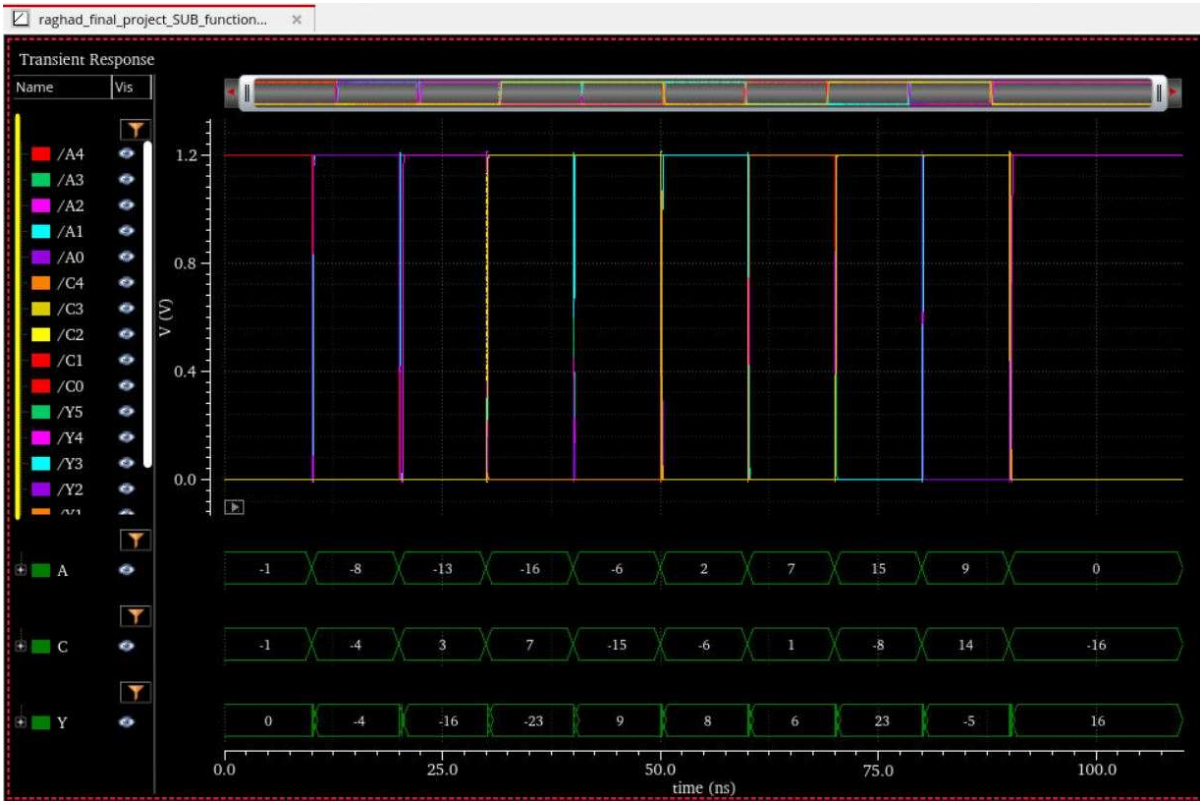


Figure 15 — Transient simulation of the 5-bit subtractor at VDD = 1.2 V. The A, C and Y buses are shown decoded as signed decimal values.

A	C	Expected A − C	Measured Y	Result
−1	−1	0	0	Correct
−8	−4	−4	−4	Correct
−13	3	−16	−16	Correct
−16	7	−23	−23	Correct
−6	−15	9	9	Correct
2	−6	8	8	Correct
7	1	6	6	Correct
15	−8	23	23	Correct
9	14	−5	−5	Correct
0	−16	16	16	Correct

Table 2 — Vectors read from the 5-bit subtractor simulation. All ten cases match the expected signed result.

Two things are worth noting in this table. First, the block handles the extreme operands of the signed 5-bit range correctly, including A = −16 and C = −16, which are the cases where the inversion of C plus the carry-in has to produce a carry out of the most significant bit. Second, the results reach −23 and +23, values that no longer fit in five bits — they are only representable because the carry-out of the adder is used as the sixth bit of Y. This is exactly why the output register was sized to 6 bits in the planning stage, and the simulation confirms that nothing is truncated.

Full ALU

Finally the complete ALU was verified with the same methodology: pulse generators drive the three 4-bit operands and the clock, and each of the six output bits is loaded with 10 fF. A relatively slow clock (50 MHz) was deliberately chosen for the functional test so that no timing violation could mask a logical error.

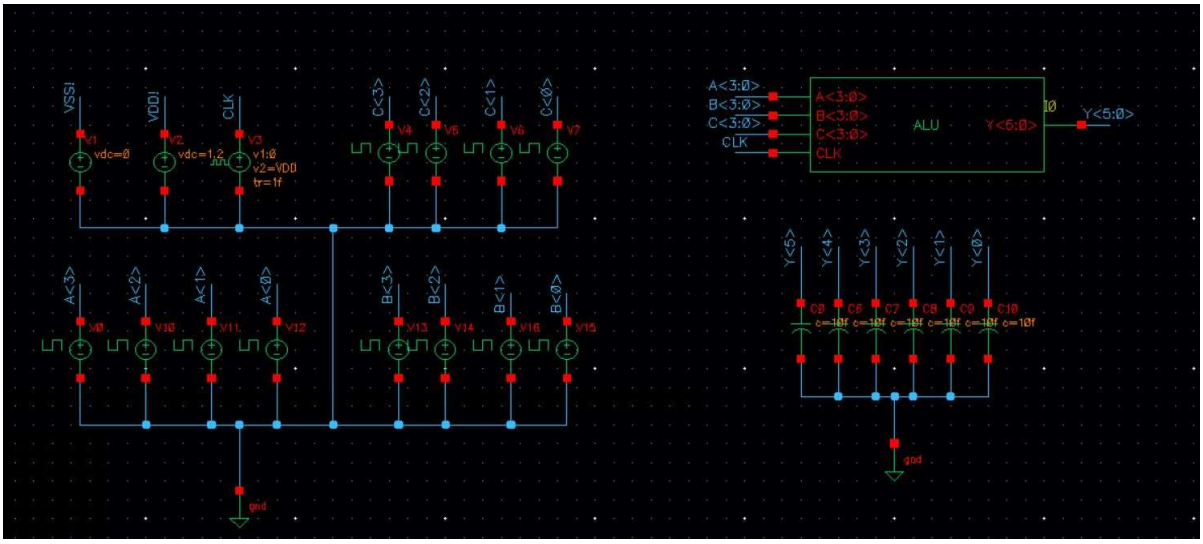


Figure 16 — Full ALU test bench — clock, three 4-bit pseudo-random operand generators and 10 fF loads on Y[5:0].

The transient simulation below shows the buses A, B, C, the intermediate result X and the output Y decoded as signed decimal values. Reading the waveform, X follows A + B one cycle later and Y follows X − C one further cycle later, exactly as expected from the pipeline relation $Y_n = (A_{n-2} + B_{n-2}) - C_{n-1}$.



Figure 17 — Full ALU transient simulation at VDD = 1.2 V. The A, B, X, C and Y buses are shown decoded as signed decimal values.

A few sample vectors taken directly from the waveform confirm both the addition and the subtraction stage, including negative operands and overflow into the extra sign bits:

A	B	X = A + B	C	Y = X − C	Result
3	4	7	−2	9	Correct
7	1	8	−3	11	Correct
6	6	12	−4	16	Correct
2	5	7	−7	14	Correct
−2	−3	−5	1	−6	Correct
−8	−1	−9	2	−11	Correct
−4	−4	−8	3	−11	Correct
−7	−6	−13	−2	−11	Correct
−1	−1	−2	7	−9	Correct

Table 3 — Sample vectors read from the ALU functional simulation. All nine cases match the expected signed arithmetic result.

The design covers the full signed range: the largest positive sum reached is X = 12 (which needs the fifth bit of X) and the largest output magnitude is Y = 16, which needs the sixth bit of Y. This confirms that the 5-bit intermediate register and the 6-bit output register are correctly sized and that no result is truncated.

Taken together, the three simulations verify the design bottom-up: the adder on its own, the subtractor on its own, and then the two of them together with the registers inside the pipelined ALU. Any error seen at the top level could therefore have been traced back to a specific block — and none was found.

5. Timing Characterisation

To find the maximal clock frequency of the ALU, three quantities have to be measured: the clock-to-Q delay of the registers, the propagation delay of the slowest combinational block, and the setup time of the registers. All delays are measured between the 50 % crossings of the input and output signals (600 mV at VDD = 1.2 V, 450 mV at VDD = 0.9 V).

Register clock-to-Q delay (T_{CQ})

T_{CQ} is the time it takes, assuming no timing violation, for the output Q to become stable after the rising edge of the clock. It was measured on a bit of the 4-bit register for both a rising and a falling output transition.

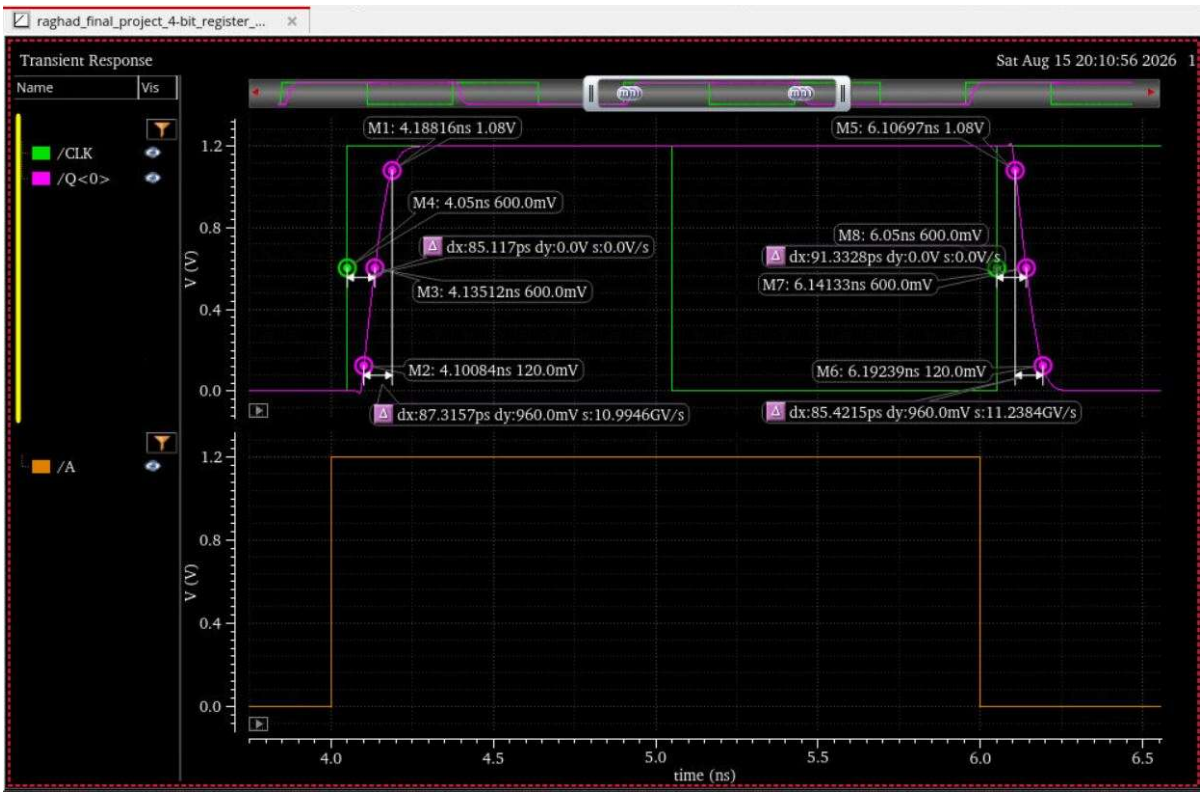


Figure 18 — Register clock-to-Q delay at VDD = 1.2 V. Rising edge: 4.05 ns → 4.135 ns = 85.1 ps. Falling edge: 6.05 ns → 6.141 ns = 91.3 ps.

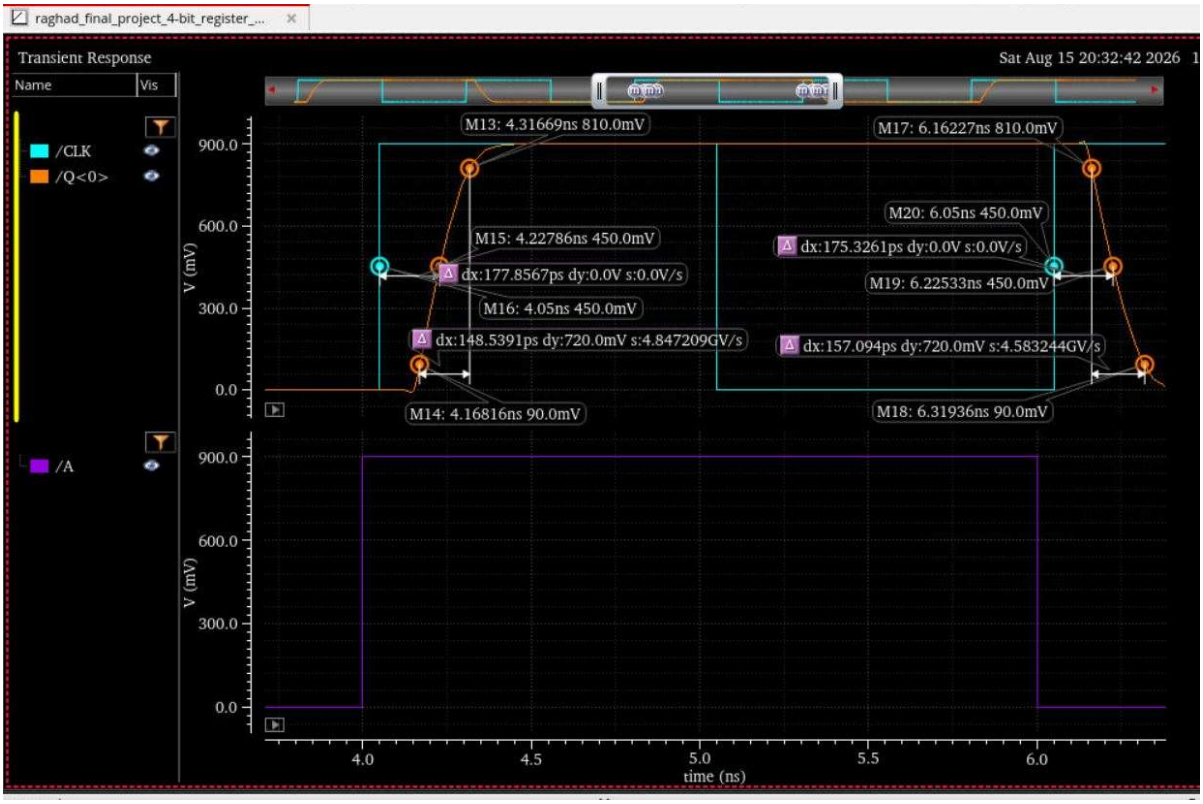


Figure 19 — Register clock-to-Q delay at VDD = 0.9 V. Rising edge: 4.05 ns → 4.228 ns = 177.9 ps. Falling edge: 6.05 ns → 6.225 ns = 175.3 ps.

Register setup time (T_{setup})

T_{setup} is how long the data has to be stable before the rising clock edge for the flip-flop to latch it correctly. It was measured by fixing the data transition at t = 4.0 ns and sweeping the delay of the clock edge, then finding the smallest delay for which the register still latches the new value.

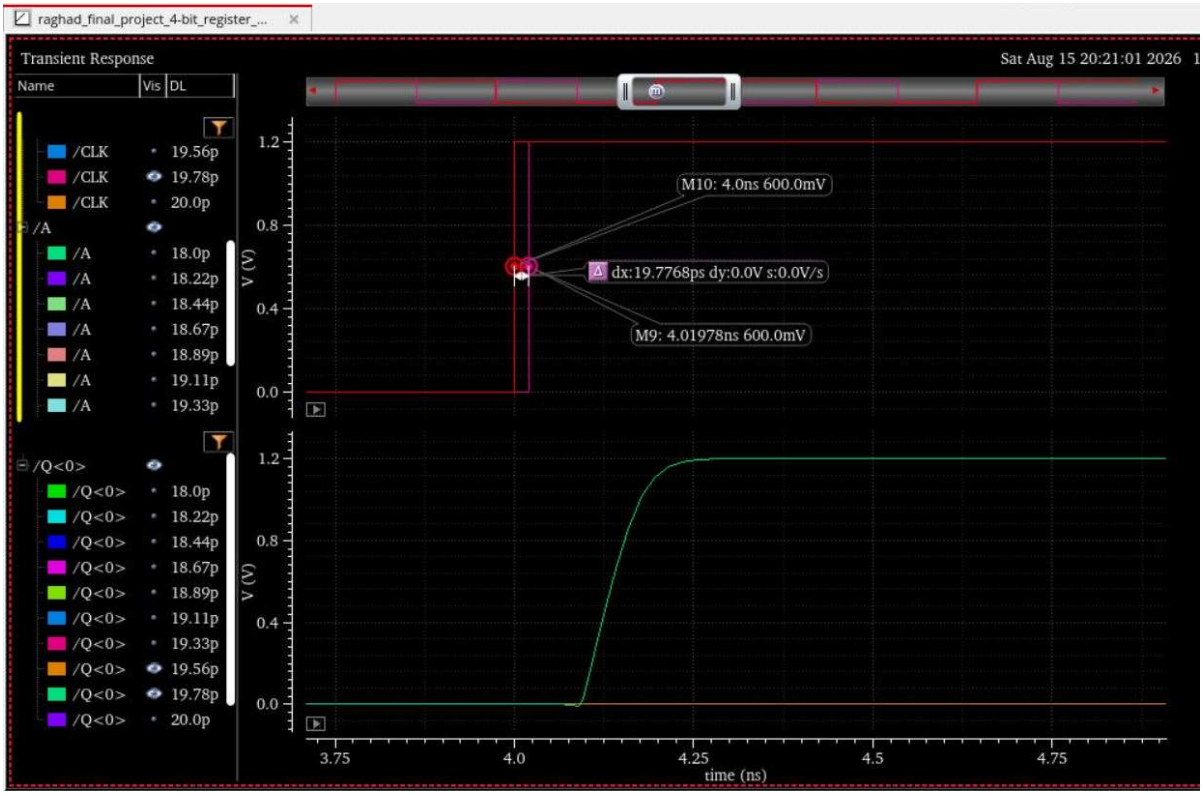


Figure 20 — Setup-time sweep at VDD = 1.2 V. For a clock delay of 19.56 ps the data is not latched, for 19.78 ps it is → T_{setup} = 0.0198 ns.

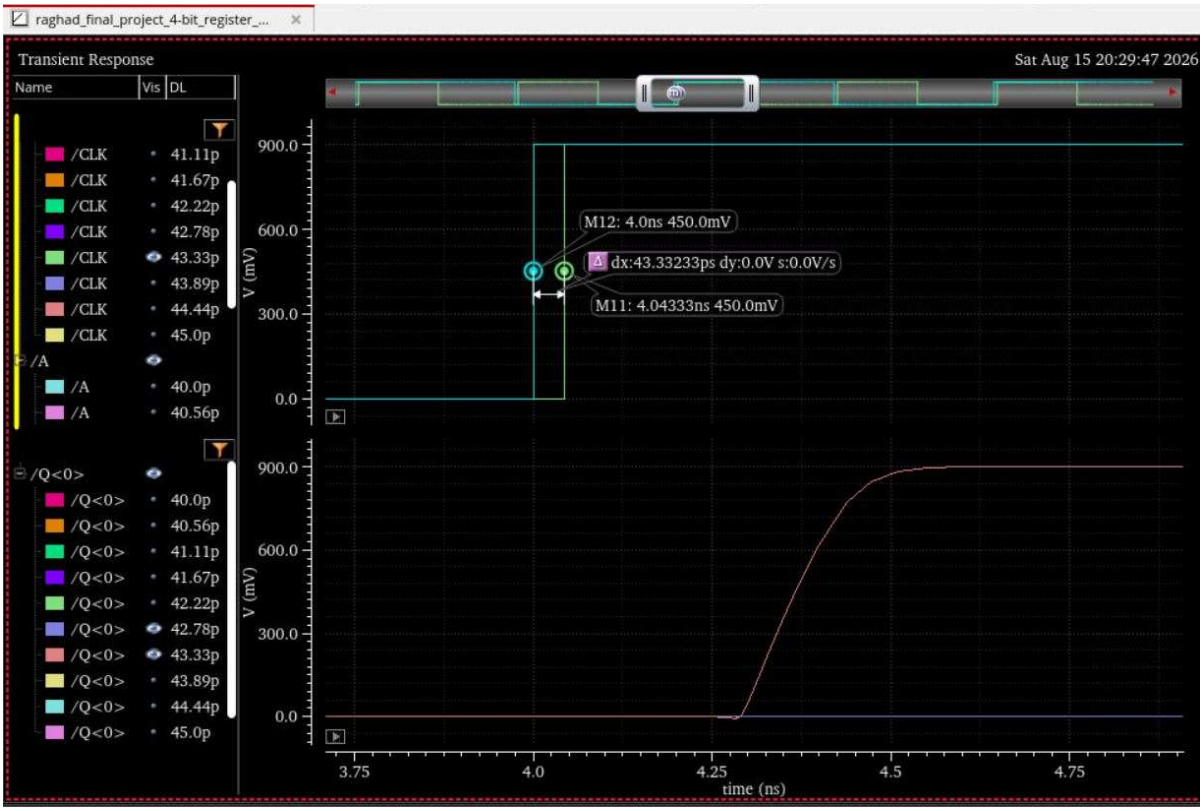


Figure 21 — Setup-time sweep at VDD = 0.9 V. For a clock delay of 42.78 ps the data is not latched, for 43.33 ps it is → T_{setup} = 0.0433 ns.

VDD	T_{CQ} rise [ps]	T_{CQ} fall [ps]	T_{CQ} worst [ns]	T_{setup} [ns]
1.2 V	85.1	91.3	0.0913	0.0198
0.9 V	177.9	175.3	0.1779	0.0433

Table 4 — Measured register timing parameters (DFFHQX1) at both supply voltages.

4-bit adder delay — schematic vs. extracted layout

The delay of the 4-bit Kogge–Stone adder was measured from a transition on the A bus to the corresponding transition on each output, first on the schematic and then on the layout after parasitic extraction with Quantus QRC. Both rising and falling transitions were measured on each of the five outputs.

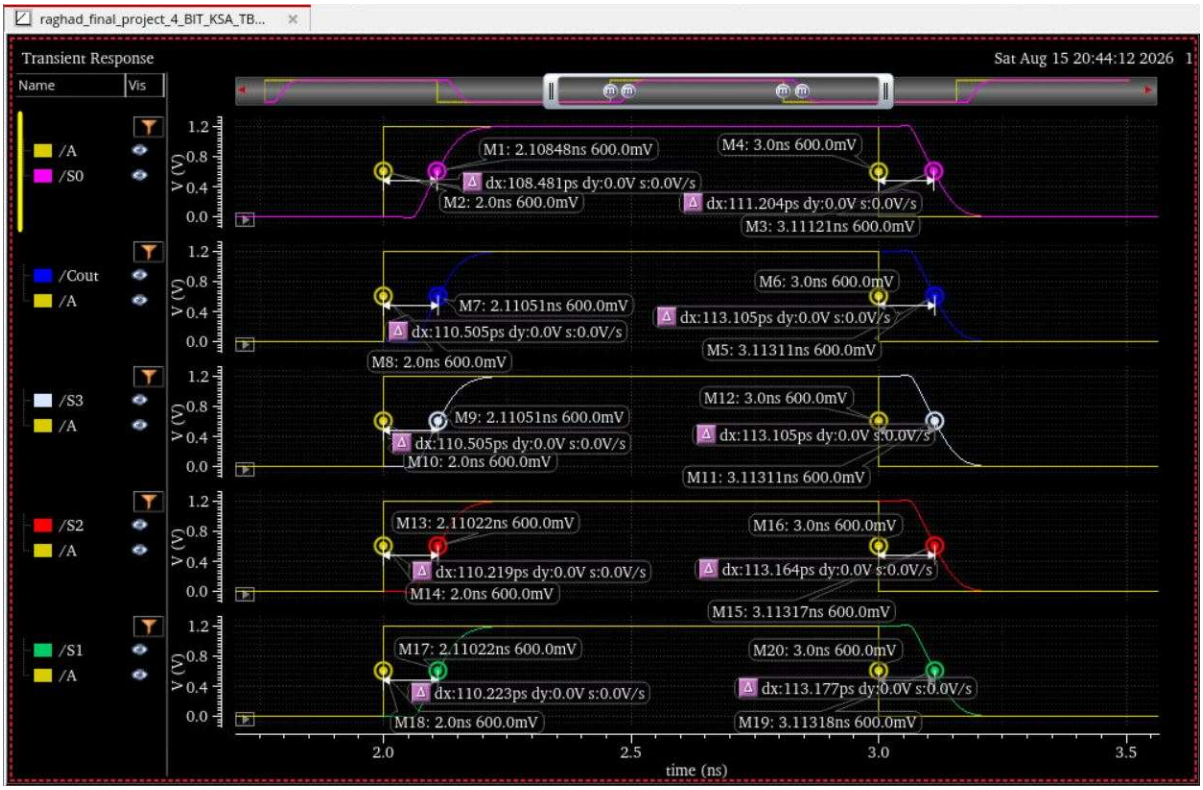


Figure 22 — Schematic-level delays of the 4-bit adder at VDD = 1.2 V.



Figure 23 — Schematic-level delays of the 4-bit adder at VDD = 0.9 V.

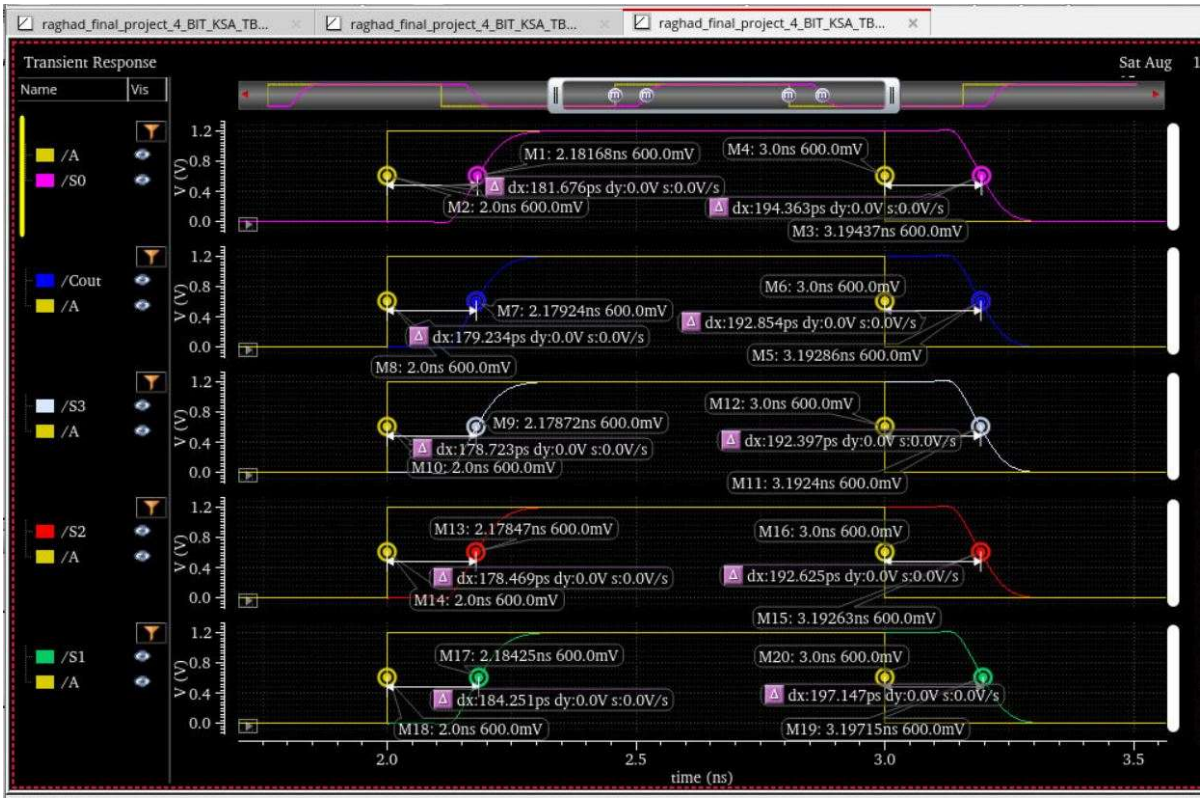


Figure 24 — Post-extraction (layout) delays of the 4-bit adder at VDD = 1.2 V.

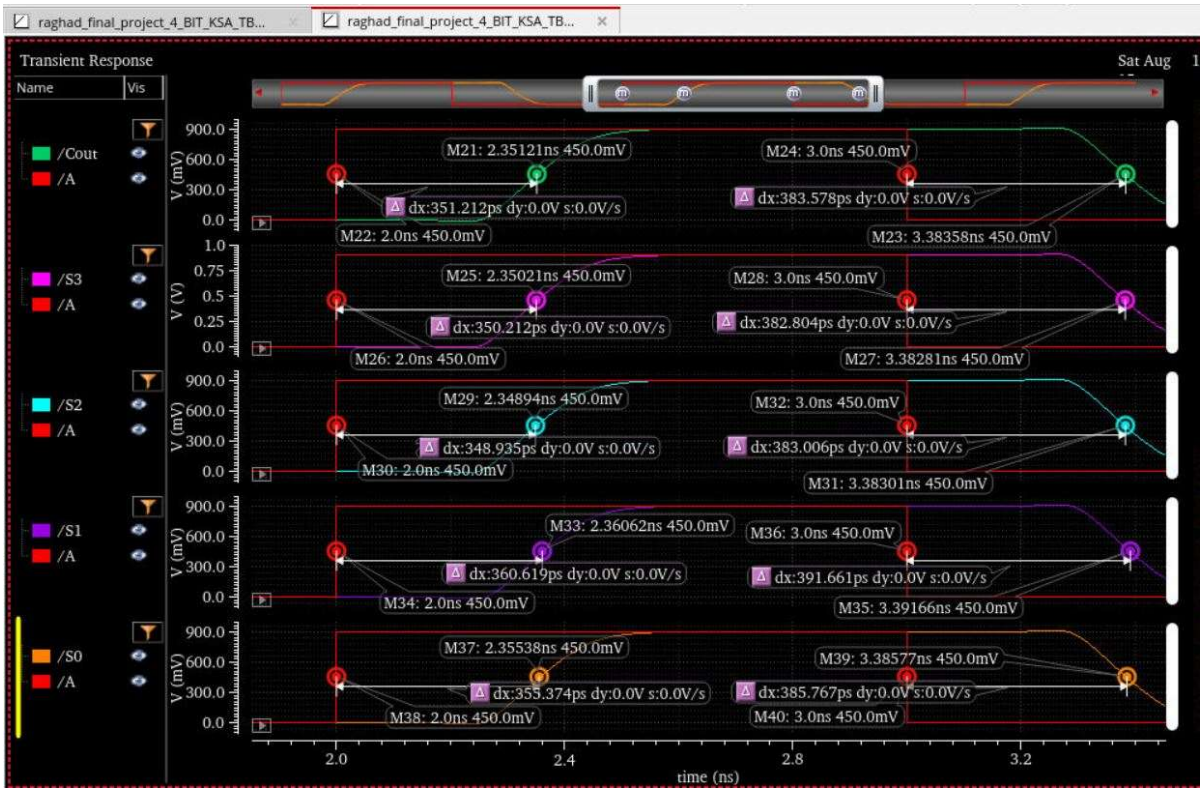


Figure 25 — Post-extraction (layout) delays of the 4-bit adder at VDD = 0.9 V.

Output	Schem. 1.2 V rise / fall [ps]	Schem. 0.9 V rise / fall [ps]	Layout 1.2 V rise / fall [ps]	Layout 0.9 V rise / fall [ps]
S0	108.5 / 111.2	211.3 / 213.5	181.7 / 194.4	355.4 / 385.8
S1	110.2 / 113.2	214.4 / 217.7	184.3 / 197.1	360.6 / 391.7
S2	110.2 / 113.2	214.4 / 217.6	178.5 / 192.6	348.9 / 383.0
S3	110.5 / 113.1	215.0 / 217.5	178.7 / 192.4	350.2 / 382.8
Cout	110.5 / 113.1	215.0 / 217.5	179.2 / 192.9	351.2 / 383.6
Worst case	**113.2**	**217.7**	**197.1**	**391.7**

Table 5 — 4-bit Kogge–Stone adder delays, schematic vs. extracted layout, at both supplies.

Two effects are visible. First, lowering the supply from 1.2 V to 0.9 V roughly doubles the delay of the block (113 ps → 218 ps on the schematic). Second, the interconnect parasitics extracted from the layout add a further 74 % at 1.2 V (113 ps → 197 ps) and 80 % at 0.9 V (218 ps → 392 ps). Both effects together mean that the extracted 0.9 V delay is about 3.5 times the ideal schematic delay at 1.2 V, which shows how important post-layout verification is.

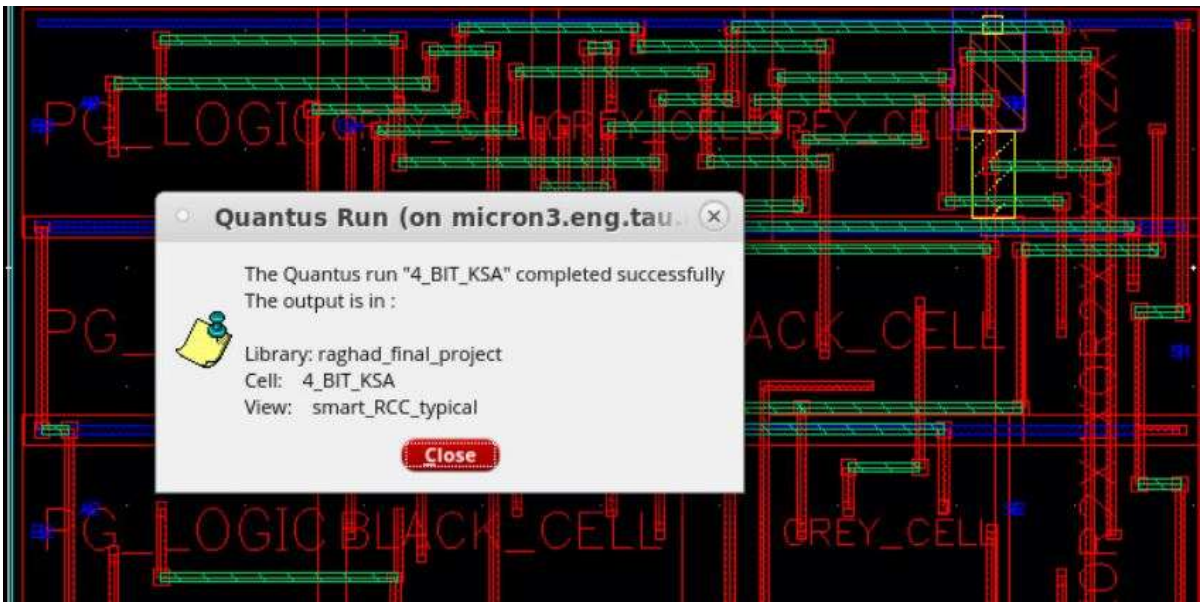


Figure 26 — Successful Quantus QRC extraction run of the 4_BIT_KSA layout (smart_RCC_typical view), used for the post-layout simulations above.

Critical path — the 5-bit subtractor

The ALU contains two register-to-register combinational paths:

- Path 1: registers A, B → 4-bit Kogge–Stone adder → register X.
- Path 2: registers X, C → 5-bit inverter → 5-bit Kogge–Stone adder → register Y.

Path 2 is the critical path. It is one bit wider, so its prefix network has three levels instead of two, and it also contains the extra inverter stage in front of the adder. The worst case inside this block is the carry that has to travel all the way from bit 0 to the most significant output, which is exactly the situation exercised by the test vectors below.

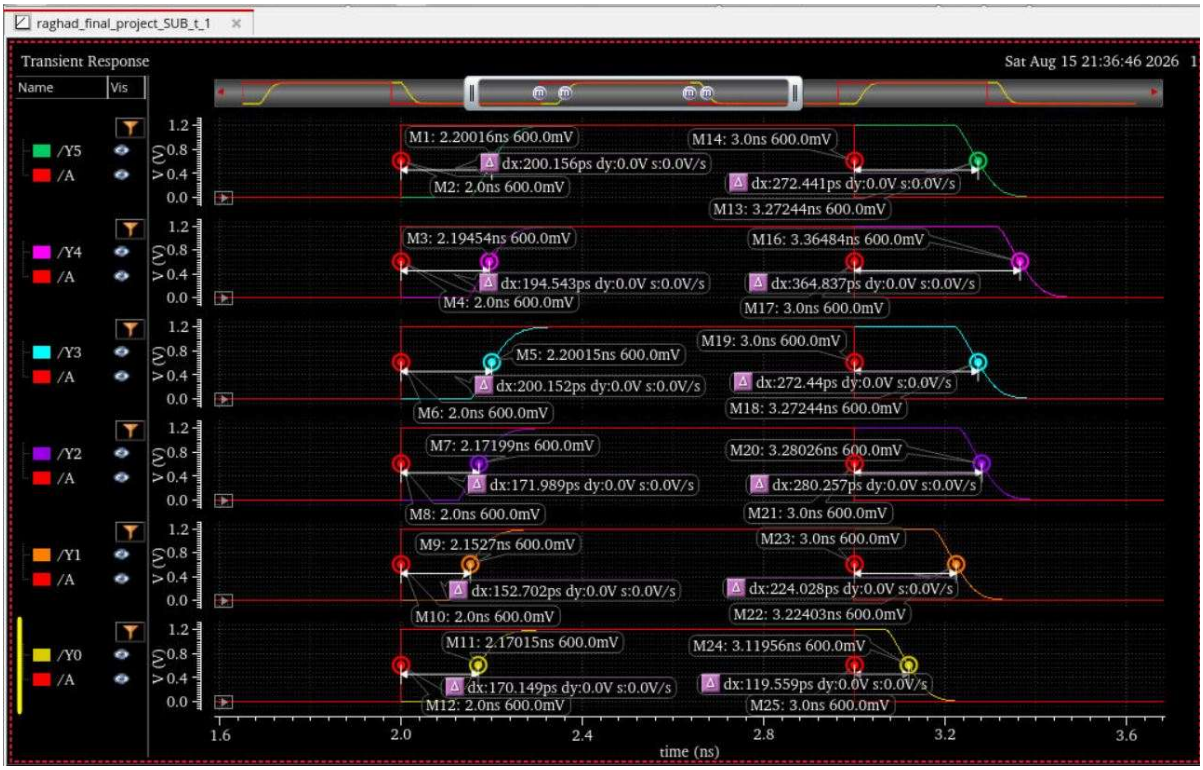


Figure 27 — Subtractor delays at VDD = 1.2 V, measured from the input transition to each output Y0–Y5. The worst case is Y4 falling, 364.8 ps.

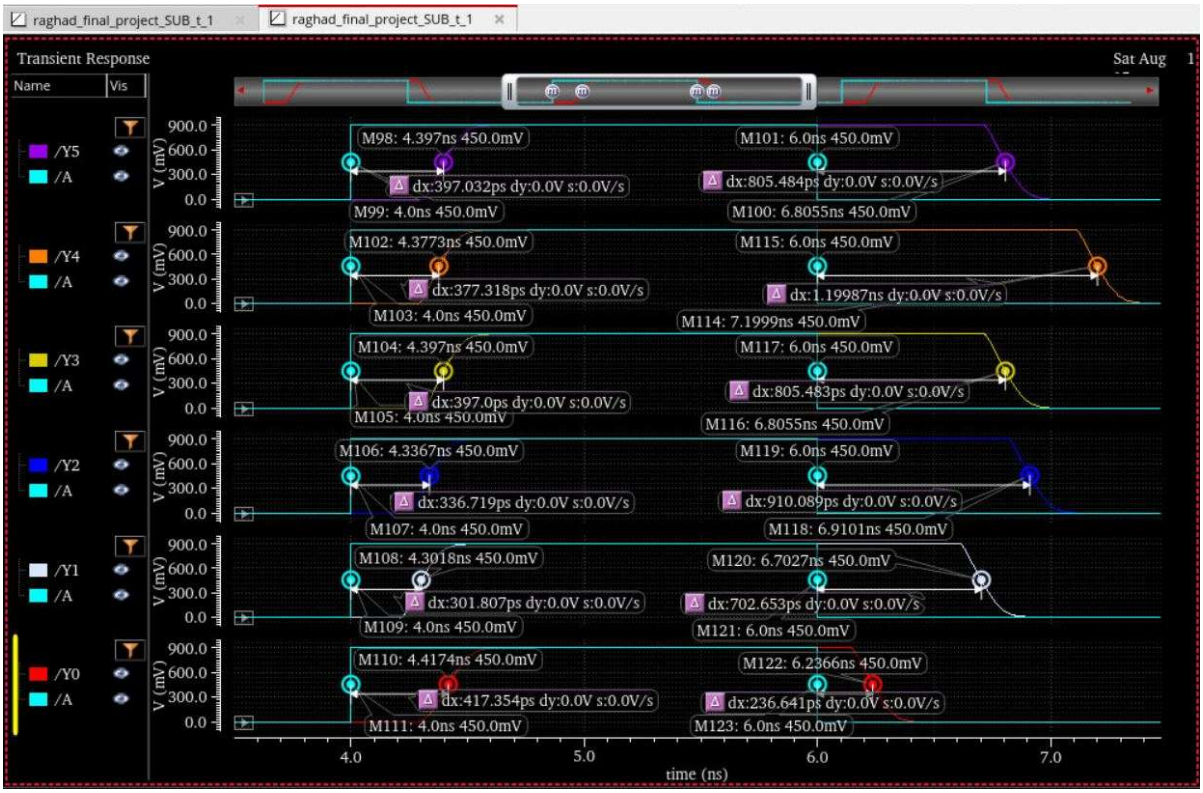


Figure 28 — Subtractor delays at VDD = 0.9 V. The worst case is again Y4 falling, now 1199.9 ps — more than three times slower.

Output	1.2 V rise [ps]	1.2 V fall [ps]	0.9 V rise [ps]	0.9 V fall [ps]
Y0	170.1	119.6	417.4	236.6
Y1	152.7	224.0	301.8	702.7
Y2	172.0	280.3	336.7	910.1
Y3	200.2	272.4	397.0	805.5
Y4	194.5	364.8	377.3	1199.9
Y5	200.2	272.4	397.0	805.5
T_{logic} (worst case)	**–**	**364.8**	**–**	**1199.9**

Table 6 — Delay of the 5-bit subtractor (the critical path of the ALU) at both supplies.

The total delay of the critical path is therefore:

$$T_{logic}\ (1.2\ V) = 0.365\ ns \qquad T_{logic}\ (0.9\ V) = 1.200\ ns$$

Maximum clock frequency

For a synchronous register-to-register path the clock period must satisfy the maximum-delay (setup) constraint:

$$T_{CLK} \geq T_{CQ} + T_{logic} + T_{setup}$$

Substituting the values measured in Sections 5.1, 5.2 and 5.4:

VDD	T_{CQ} [ns]	T_{logic} [ns]	T_{setup} [ns]	$T_{CLK,min}$ [ns]	f_{max}
1.2 V	**0.0913**	**0.3648**	**0.0198**	**0.4759**	**2.10 GHz**
0.9 V	**0.1779**	**1.1999**	**0.0433**	**1.4211**	**704 MHz**

Table 7 — Minimal clock period and maximal clock frequency of the ALU at both supply voltages.

The effect of the supply voltage on the maximal clock frequency

Reducing the supply voltage from 1.2 V to 900 mV — a 25 % reduction — costs the circuit about two thirds of its speed. The minimal clock period grows from 0.476 ns to 1.421 ns, a factor of almost three, so the maximal clock frequency falls from 2.10 GHz to 704 MHz.

The reason is that the drive current of a MOS transistor scales roughly with $(V_{DD} - V_{th})^2$ while the capacitance that has to be charged stays the same. At 0.9 V the overdrive voltage is much smaller, so each gate takes considerably longer to charge the next stage. The effect is not spread evenly over the design: the register parameters roughly double (T_{CQ}: 0.091 ns → 0.178 ns), while the deep logic of the critical path suffers far more (T_{logic}: 0.365 ns → 1.200 ns, a factor of 3.3), because the degradation of every individual gate accumulates along the chain of the prefix network.

The design remains logically correct at both supplies — the functional simulation passes in both cases — so this is a pure performance/power trade-off: 0.9 V operation cuts the dynamic power (which scales with V_{DD}^2, i.e. to about 56 % of the 1.2 V value) at the price of roughly a third of the throughput.

6. Hierarchical Floor Plan

The design was entered as a strict hierarchy, and the same hierarchy was kept in the layout so that each block can be verified, characterised and reused independently.

The physical floor plan follows the data flow of the design from left to right. The three input registers are placed on the left, immediately next to the 4-bit adder; the intermediate register X sits between the two arithmetic stages; the subtractor is placed to its right, with the 5-bit inverter directly in front of the 5-bit adder so that the inverted C bus does not have to travel far; and the 6-bit output register closes the chain on the right. Keeping the registers adjacent to the logic they feed keeps the clock-to-Q and setup paths short, which is what allows the clock periods reported in Table 10 to be met.

Inside the adder itself the floor plan is row based. The cells are organised in four abutted standard-cell rows, with the VDD and VSS rails running horizontally and shared between adjacent rows, exactly like an automatically placed standard-cell block. Each row holds one bit slice of the adder: its PG_LOGIC cell, the black or grey cell of the prefix network for that bit, and the post-processing XOR2X1 gate at the right-hand edge, so that signals flow from left to right within a row and vertically only where the prefix network needs to combine two bit positions.

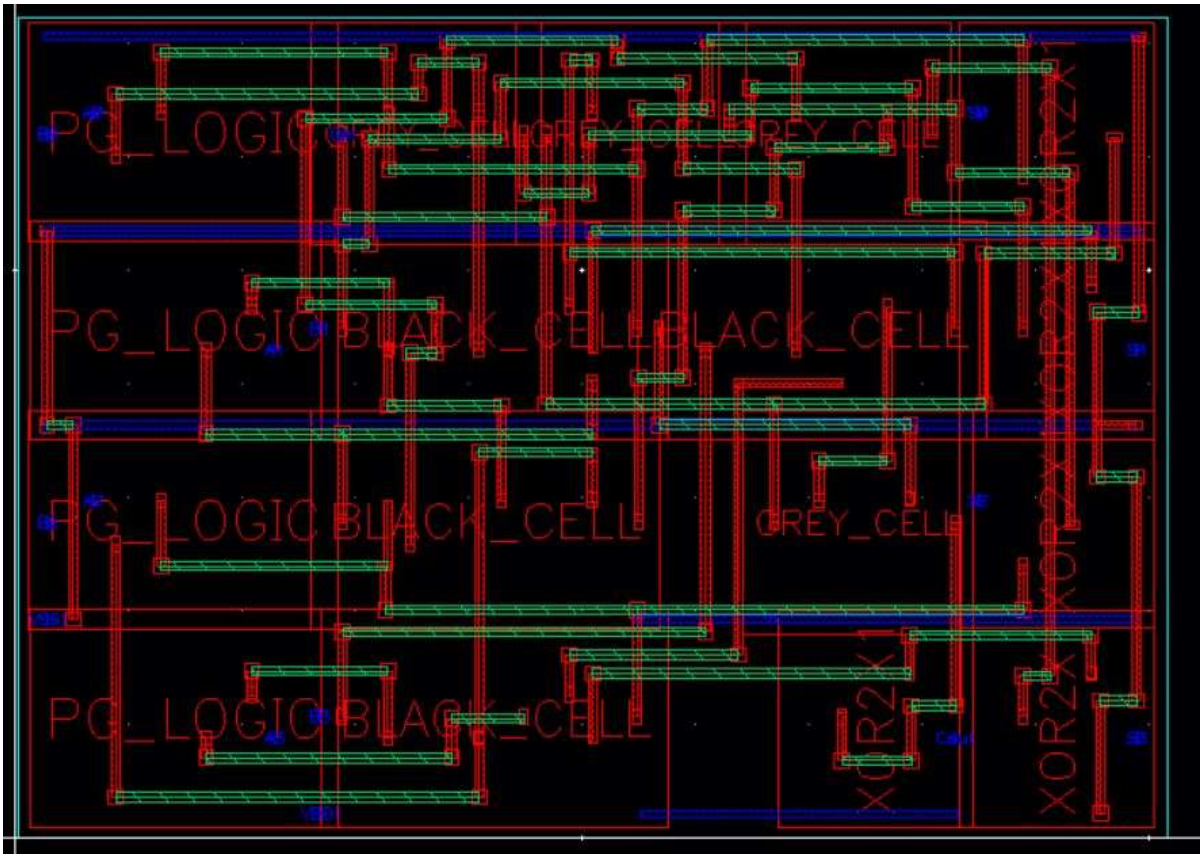


Figure 29 — Floor plan of the 4-bit adder layout — four standard-cell rows, one bit slice per row, with the PG_LOGIC, prefix (black / grey) and XOR2X1 cells arranged from left to right.

7. Layout of the Adder

As required, a full layout was built for the adder. It was assembled hierarchically: the leaf cells PG_LOGIC, GREY_CELL and BLACK_CELL were laid out first from the gsclib045 standard cells, and the 4_BIT_KSA layout was then built by instantiating and abutting those blocks in four rows.

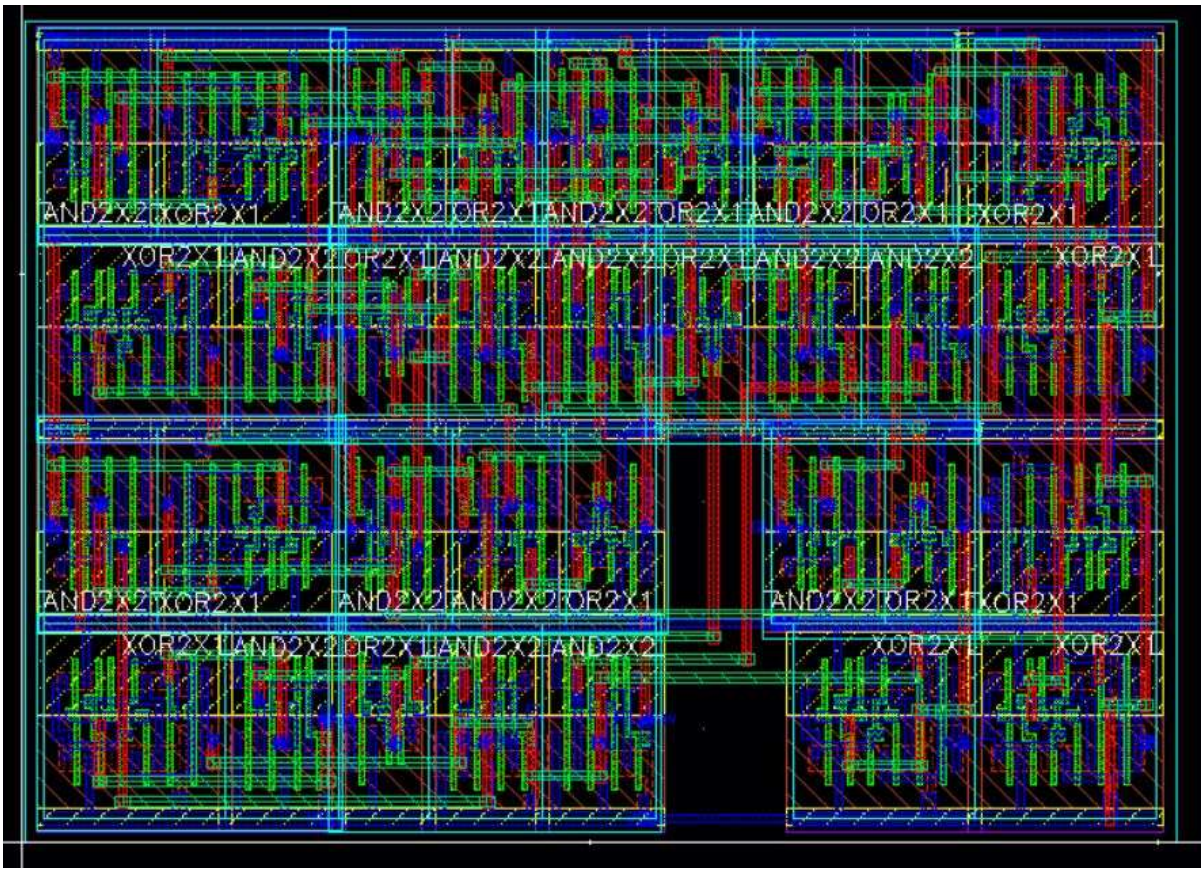


Figure 30 — Complete 4_BIT_KSA layout — all metal layers shown, with the gsclib045 cell instances (AND2X2, OR2X1, XOR2X1) visible in every row.

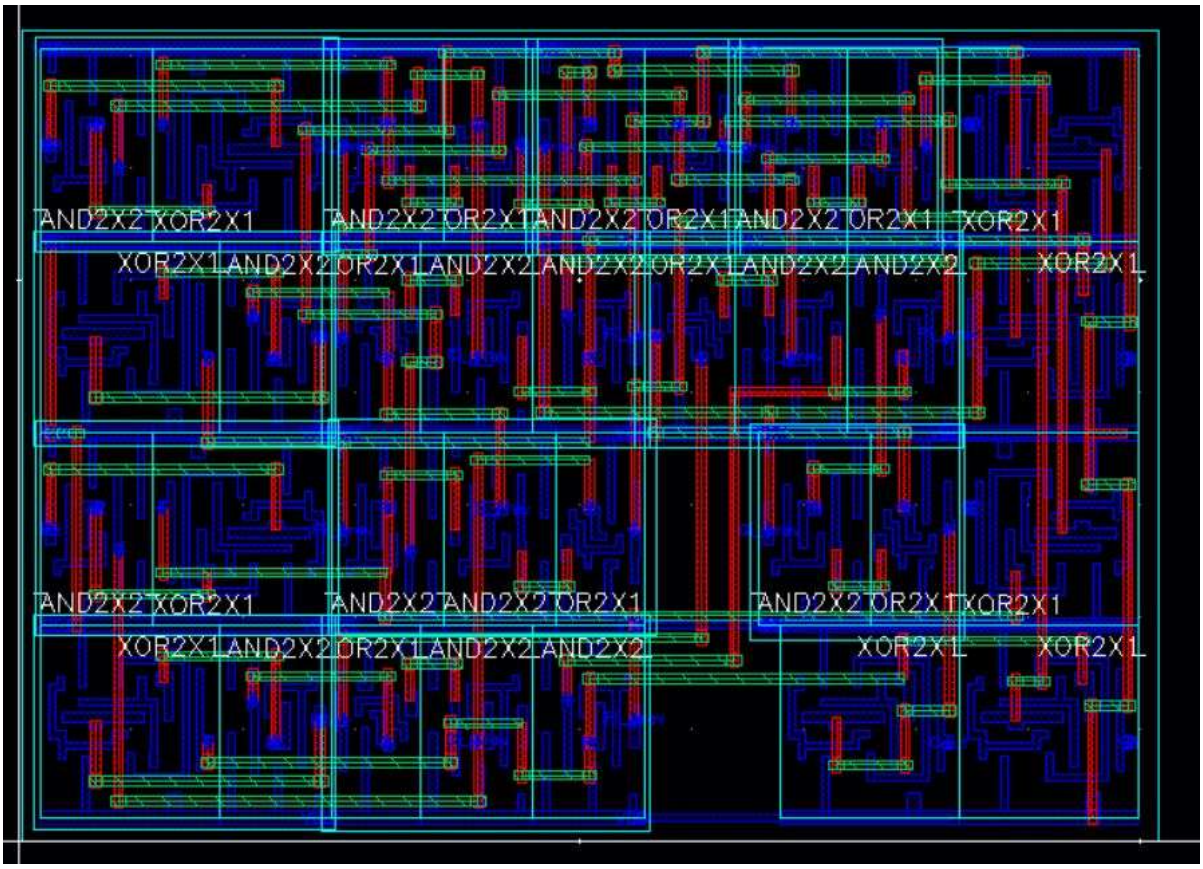


Figure 31 — The same layout with the routing layers highlighted — the inter-block connections of the prefix network are routed over the cells.

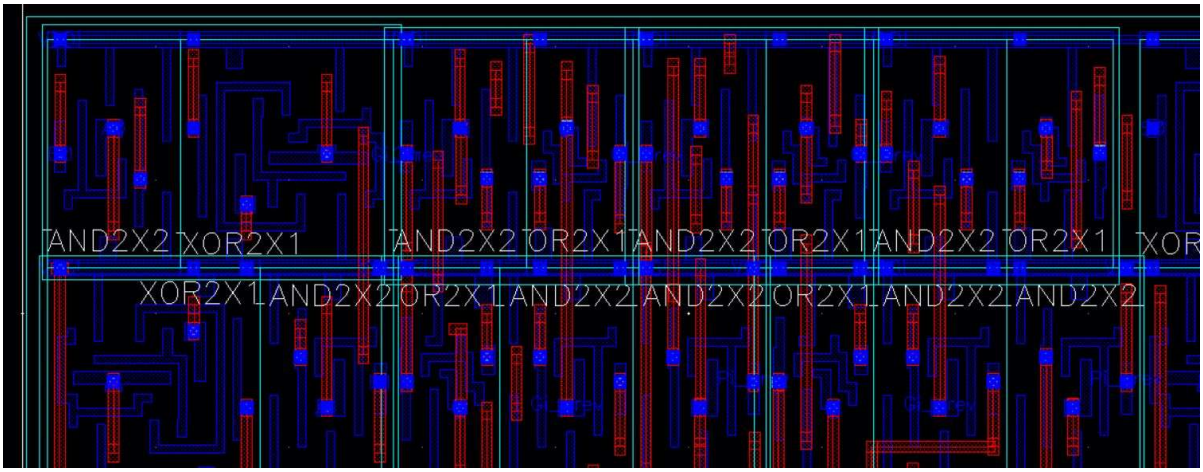


Figure 32 — Zoom on the upper rows, showing the abutted standard cells, the shared power rails and the poly / diffusion structures of the leaf cells.

Layout area and comparison with the theoretical area

The bounding box of the finished 4_BIT_KSA cell was measured with a ruler in Virtuoso:

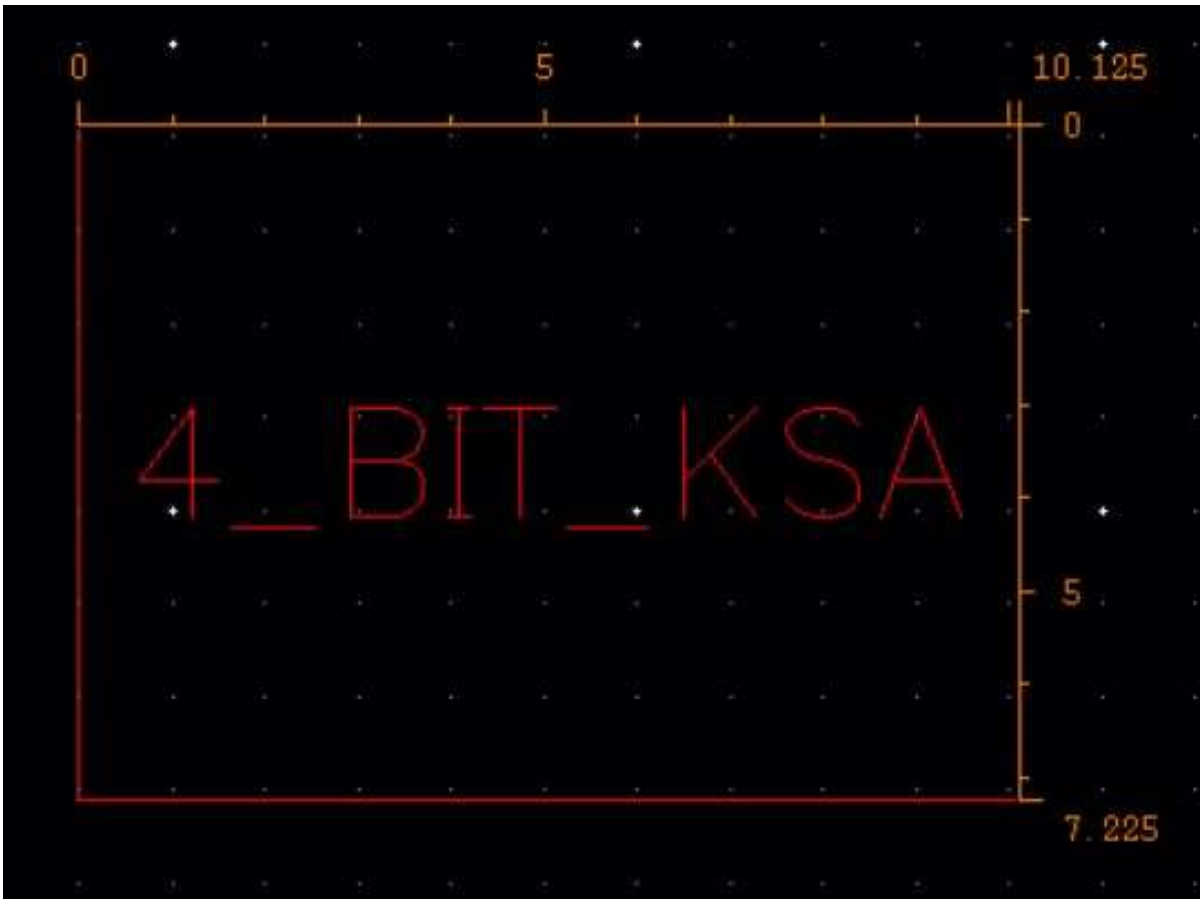


Figure 33 — Bounding box of the 4_BIT_KSA layout: 10.125 µm × 7.225 µm.

$$\textit{Layout area} = 10.125\ \mu m \times 7.225\ \mu m = 73.15\ \mu m^2$$

The theoretical standard-cell area of the same block, from Table 2, is:

$$16 \times AND2X2 + 8 \times OR2X1 + 9 \times XOR2X1 = 77.44\ \mu m^2$$

Quantity	Value
Theoretical standard-cell area	77.44 µm²
Measured layout area	73.15 µm²
Layout / theoretical ratio	94.5 %
Requirement	**≤ 150 % — met**

Table 8 — Area of the adder layout compared with the theoretical standard-cell area.

The layout occupies 94.5 % of the theoretical area, comfortably inside the requirement of not exceeding 150 %. The reason the measured area comes out slightly below the estimate is the row height: the theoretical figure uses the 1.9 µm value from the planning stage, whereas the four abutted rows of the real layout measure 7.225 µm in total, i.e. about 1.81 µm per row. The total cell width placed in the layout (4 rows × 10.125 µm ≈ 40.5 µm) matches the sum of the individual cell widths (40.76 µm) almost exactly, which confirms that the rows are fully abutted with essentially no wasted placement area and that all 33 cells of the adder are present.

8. Physical Verification — DRC and LVS

Design Rule Check

DRC was run with PVS 23.11-64b on the adder layout and on each of its sub-blocks (PG_LOGIC, GREY_CELL, BLACK_CELL) using the gpdk045 rule deck. The run report shows 562 rule checks performed on 421 geometries and, most importantly, a total of zero DRC results — that is, zero violations. The DRC Debug Environment confirms this by listing the top cell as 4_BIT_KSA (E0), with no error markers to browse. The messages listed at the bottom of the report window are informational warnings from the run script (an existing output GDS file being overwritten and a rule file that is set in the control file); they are not geometry violations.

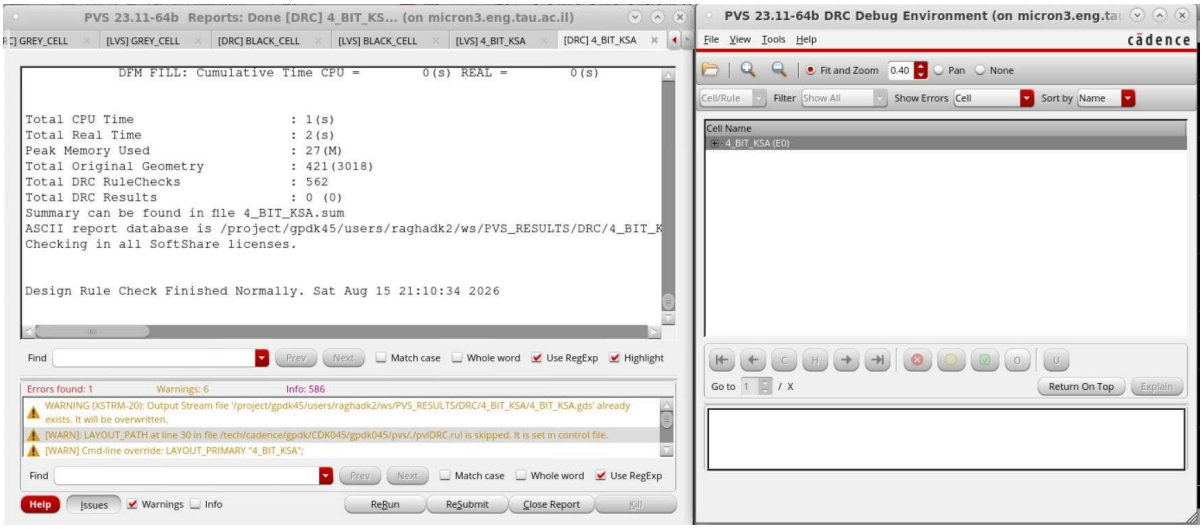


Figure 34 — PVS DRC report for the 4_BIT_KSA layout — "Total DRC Results : 0 (0)" and an empty error list in the DRC Debug Environment.

Layout Vs Schematic

LVS was then run to compare the extracted netlist of the layout against the schematic netlist. The comparison finished with a full MATCH: the ERC results are empty, the extraction is clean, and the comparison result is a match. The layout of the adder is therefore electrically identical to the schematic that was simulated in Section 4.

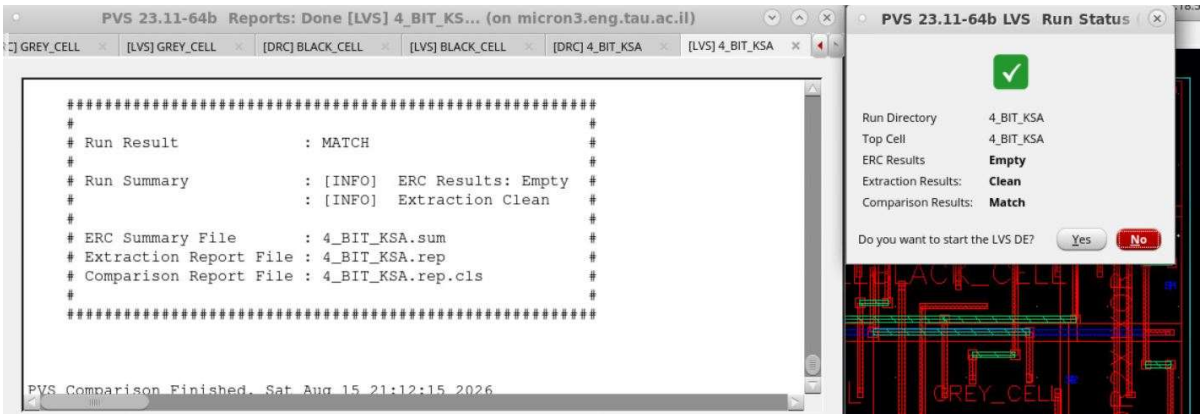


Figure 35 — PVS LVS report for the 4_BIT_KSA layout — Run Result: MATCH, ERC Results: Empty, Extraction: Clean, Comparison: Match.

9. Conclusion

In this stage the 4-bit ALU that was planned in the previous stage was fully implemented, simulated, laid out and verified. The main results are:

- The complete ALU was entered in Cadence Virtuoso as a four-level hierarchy built only from gsclib045 standard cells, comprising 105 leaf cells with a theoretical area of 329.46 μm^2.
- Functional simulation of the full ALU confirms correct signed two's-complement behaviour over the whole input range, including negative operands and results that need the extra sign bits of X[4:0] and Y[5:0].
- The critical path is the second arithmetic stage — the 5-bit inverter plus the 5-bit Kogge–Stone adder — with a measured delay of 0.365 ns at 1.2 V and 1.200 ns at 900 mV.
- The maximal clock frequency is 2.10 GHz at 1.2 V (T_{CLK} = 0.476 ns) and 704 MHz at 900 mV (T_{CLK} = 1.421 ns).
- Post-layout simulation of the adder shows that interconnect parasitics add 74–80 % to the schematic delay, confirming that extraction is essential before trusting any timing number.
- The adder layout measures 10.125 µm × 7.225 µm = 73.15 μm^2, i.e. 94.5 % of the theoretical standard-cell area — well within the 150 % limit.
- DRC and LVS both pass with zero errors, and LVS reports a full MATCH against the schematic.

The design therefore meets every functional and physical requirement of the implementation stage. The clearest engineering lesson from the measurements is how differently the two supply voltages behave: a 25 % reduction of VDD buys roughly a 44 % reduction in dynamic power but costs two thirds of the maximum operating frequency, and that cost falls almost entirely on the deep logic of the prefix network rather than on the registers.

Part 2 – Ring Oscillator Design

Calculating the required delay

We designed a ring oscillator using five inverters, meaning that the oscillator contains five stages. The oscillation frequency is given by:

$$f_{osc} = \frac{1}{2\, t_d\, n}$$

Using n = 5 and the required frequency f_{osc} = 2.5 MHz, the required delay per stage is:

$$t_d = 1 / (2\, f_{osc}\, n) = 1 / (2 \cdot 2.5 \cdot 10^6 \cdot 5) = 40\ [ns]$$

Measuring the inverter delay

The total delay of each stage consists of the intrinsic inverter delay and the additional RC delay:

$$t_d = t_{inv} + t_{RC}$$

We design this circuit:

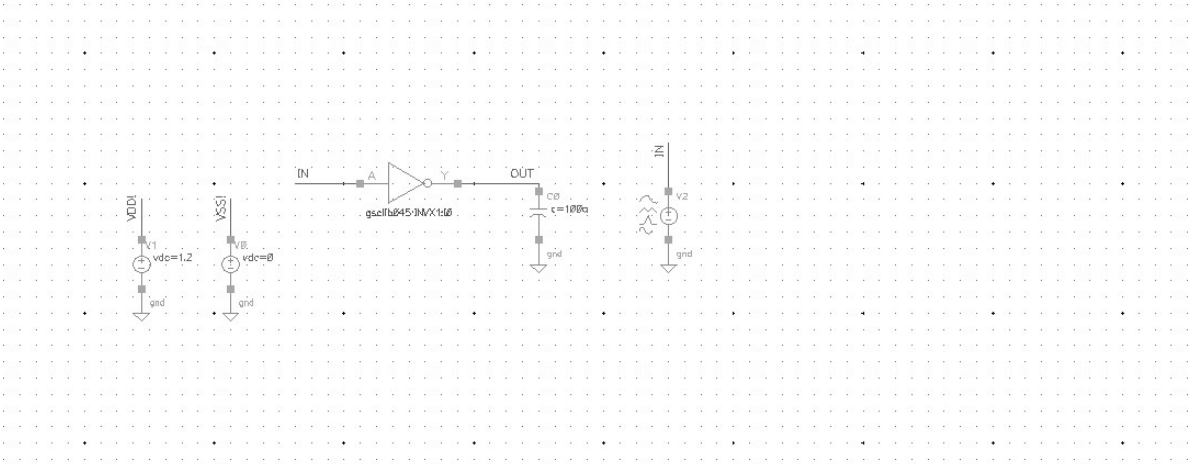


Figure 36 — Circuit used to measure the intrinsic delay of the INVX1 inverter, with a 0.1 fF load capacitor at the output.

A small load capacitor of 0.1fF was added to the inverter output. This value is much smaller than the capacitors used later, so its effect on the measured delay is negligible.

We selected an INVX1 inverter. For an inverter scaled by a factor s, the equivalent resistance becomes R_{driver}/s, while the equivalent capacitance becomes sC_{driver}. Therefore, the scaling factors cancel when calculating the intrinsic RC delay:

$$(R_{driver} / s) \cdot sC_{driver} = R_{driver}\, C_{driver}$$

As a result, the inverter size has little effect on the intrinsic delay and oscillation frequency. Although a larger inverter has a higher intrinsic capacitance, its contribution is negligible compared with the external RC delay, as shown in the following simulation.

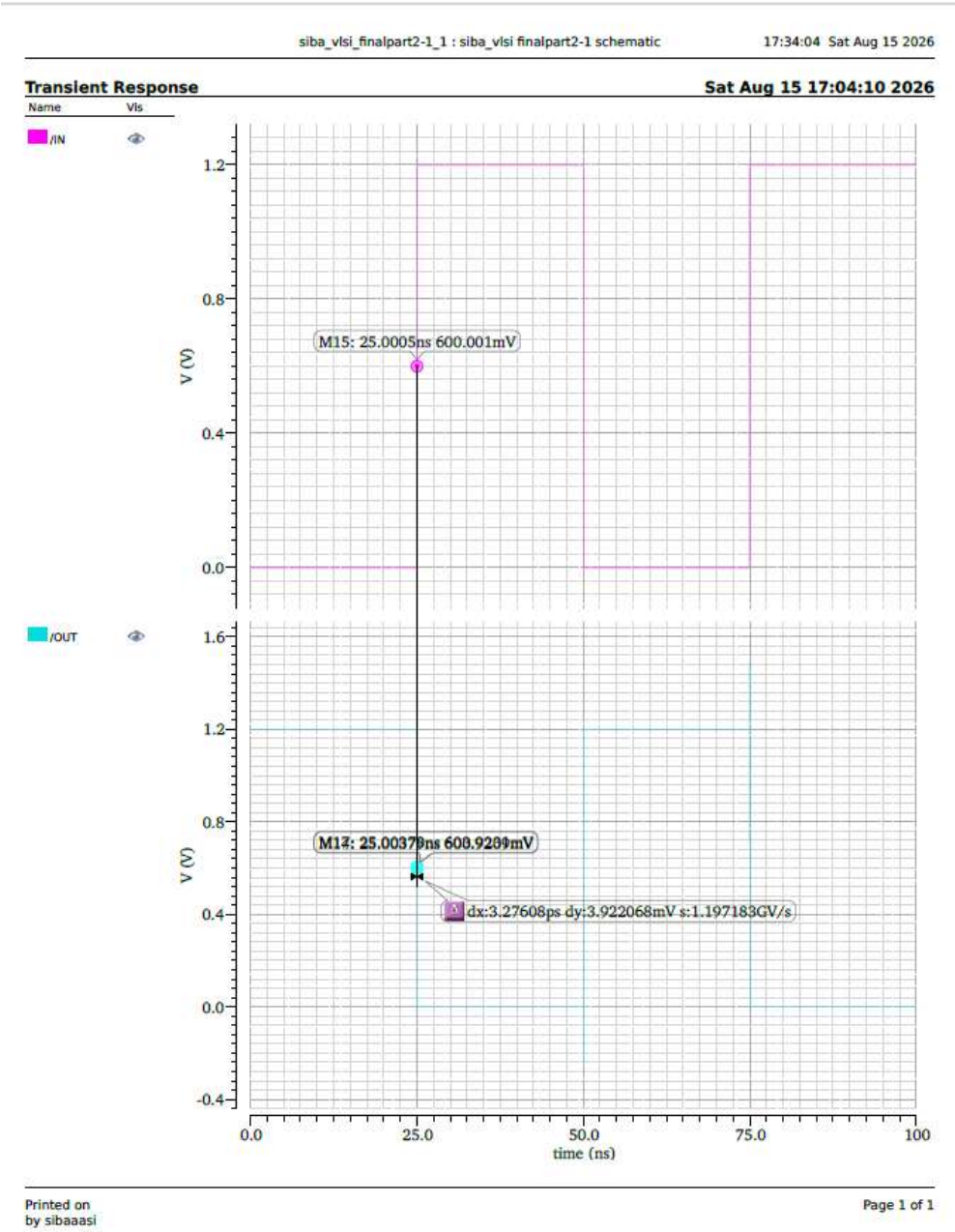


Figure 37 — Transient simulation of the inverter delay measurement. The markers give dx = 3.27608 ps between the 50 % crossings of IN and OUT.

The measured intrinsic delay of the inverter is:

$$t_{inv} = 3.28\ ps$$

Since 3.28ps ≪ 40ns, the inverter delay is negligible compared with the required delay per stage. Therefore, the RC network provides almost all the stage delay:

$$t_d = t_{inv} + t_{RC} \sim t_{RC}$$

For a lumped RC network, the propagation delay is approximated by:

$$t_{RC} = 0.69\,RC$$

Using C = 10fF and $t_{RC} \approx 40$ns:

$$0.69\,R\,(10 \times 10^{-15}) = 40 \times 10^{-9}$$

Therefore: R ≈ 5.8 MΩ

Using these values, we constructed the five-stage ring oscillator, with a 5.8MΩ resistor and a 10fF capacitor at each stage.

We assemble the ring oscillator:

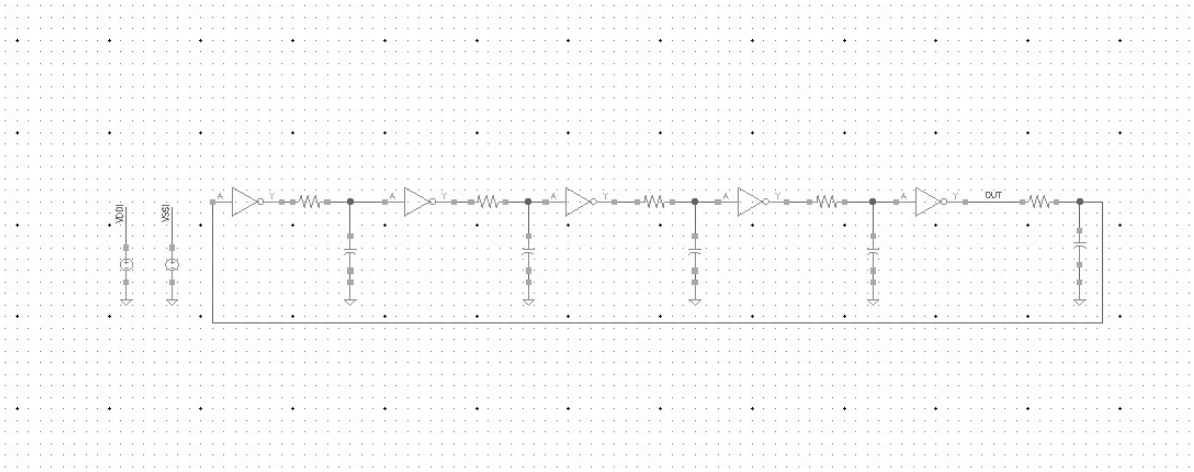


Figure 38 — The five-stage ring oscillator — five INVX1 inverters, each followed by a 5.8 MΩ series resistor and a 10 fF capacitor to ground.

We run a transient simulation and get:

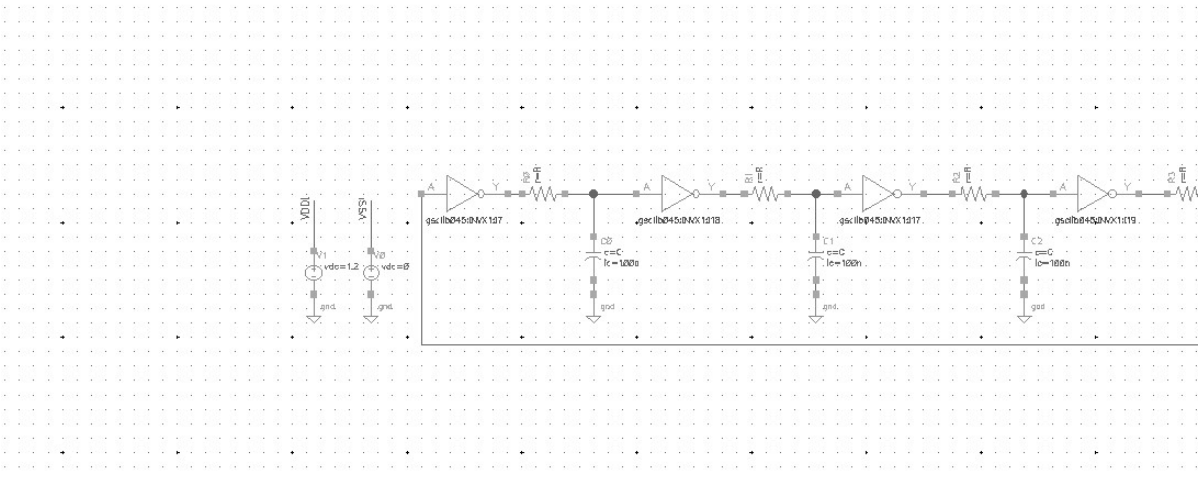


Figure 39 — Left-hand part of the ring oscillator schematic, showing the supply sources and the first RC stages.

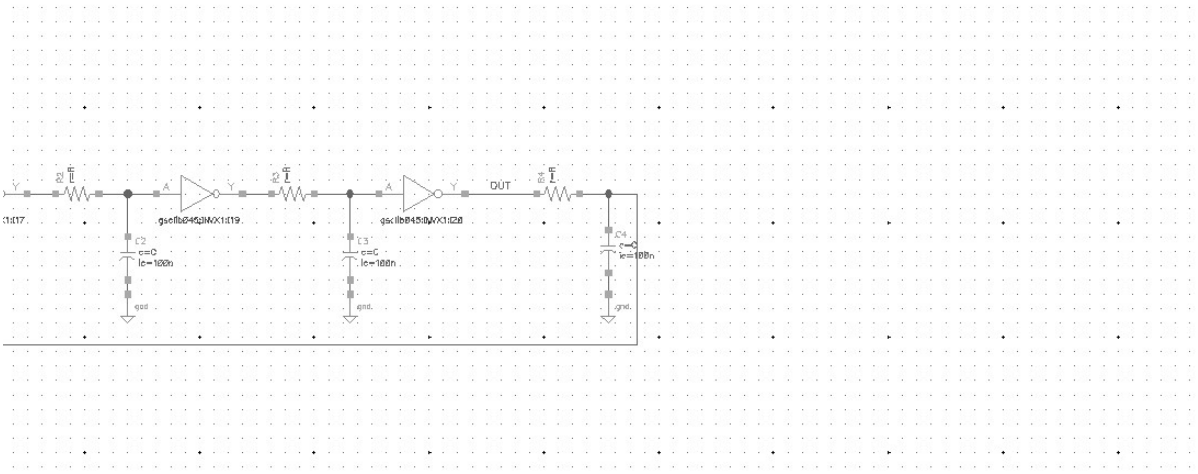


Figure 40 — Right-hand part of the ring oscillator schematic, closing the loop from the last stage back to the first.

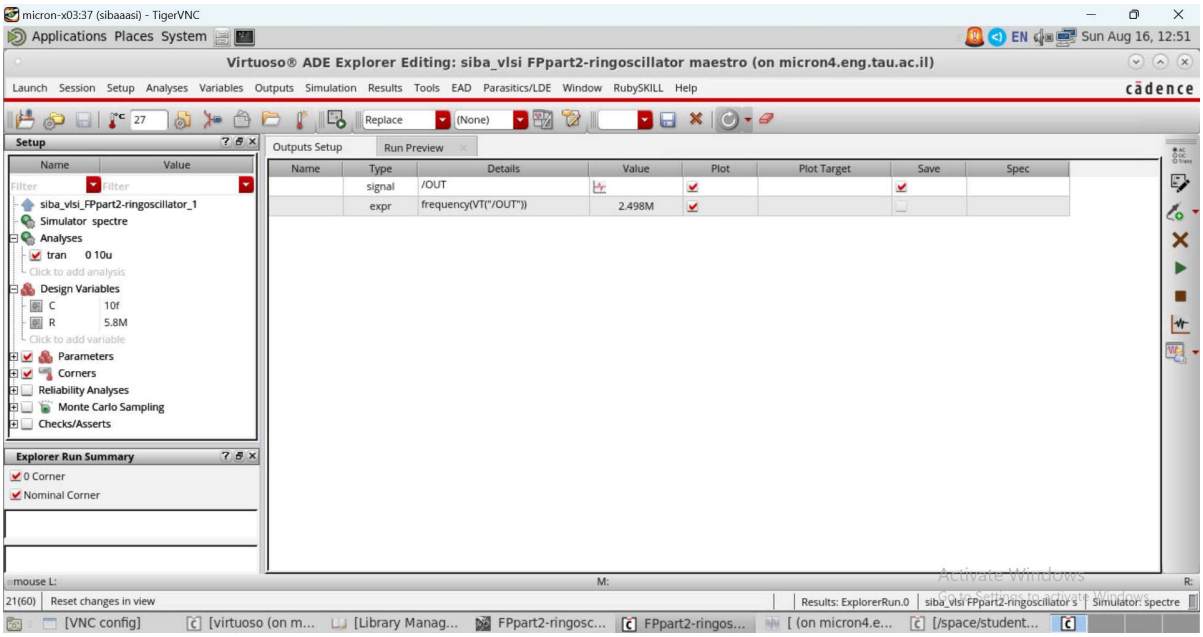


Figure 41 — ADE Explorer run preview with the design variables C = 10f and R = 5.8M. The measured frequency of /OUT is 2.498 MHz.

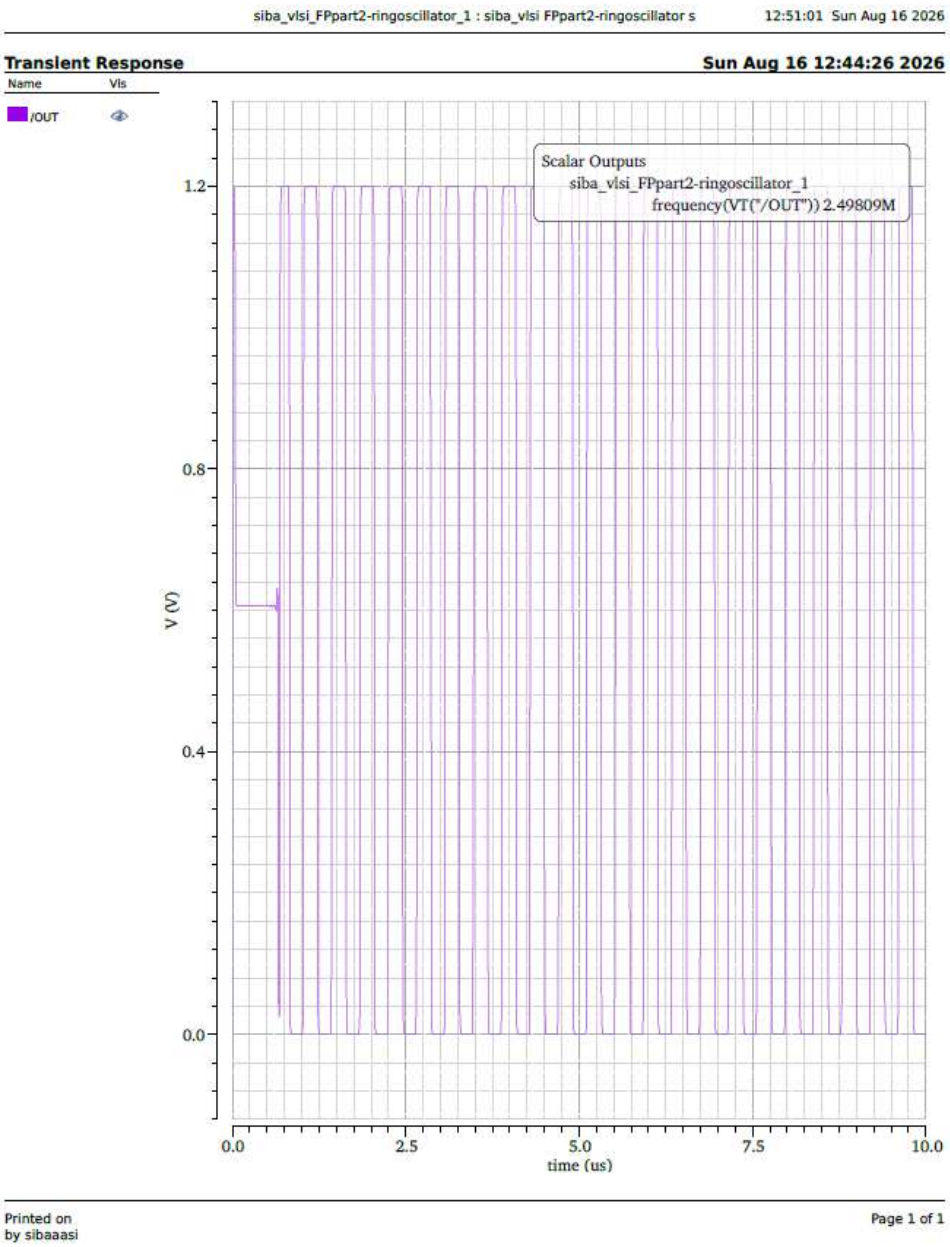


Figure 42 — Transient response of the unloaded ring oscillator over 10 µs. frequency(VT("/OUT")) = 2.49809 MHz.

The simulated oscillation frequency is f = 2.498MHz. Compared with the target frequency of 2.5MHz, the relative error is:

$$error = (2.5 - 2.498) / 2.5 \cdot 100\% = 0.08\%$$

This very small error indicates that the designed ring oscillator closely matches the required frequency.

Next, a 100fF load capacitor was connected to the output to examine its effect on the oscillator's frequency and waveform.

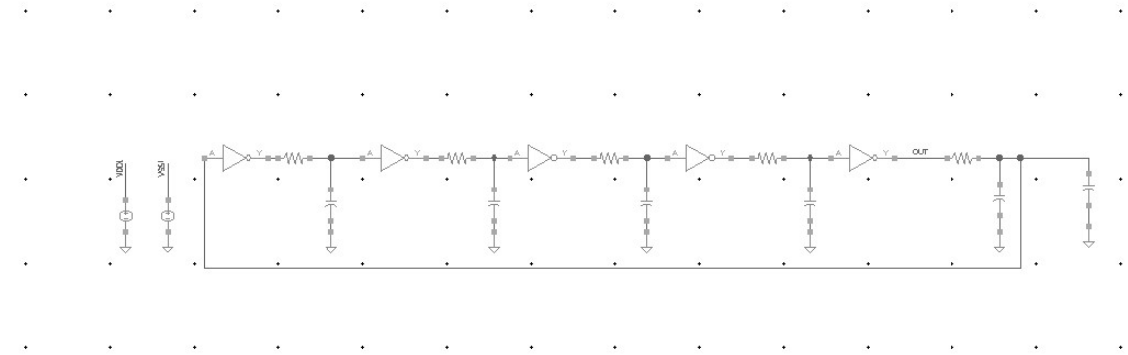


Figure 43 — The ring oscillator with the additional 100 fF load capacitor connected directly to the OUT node.

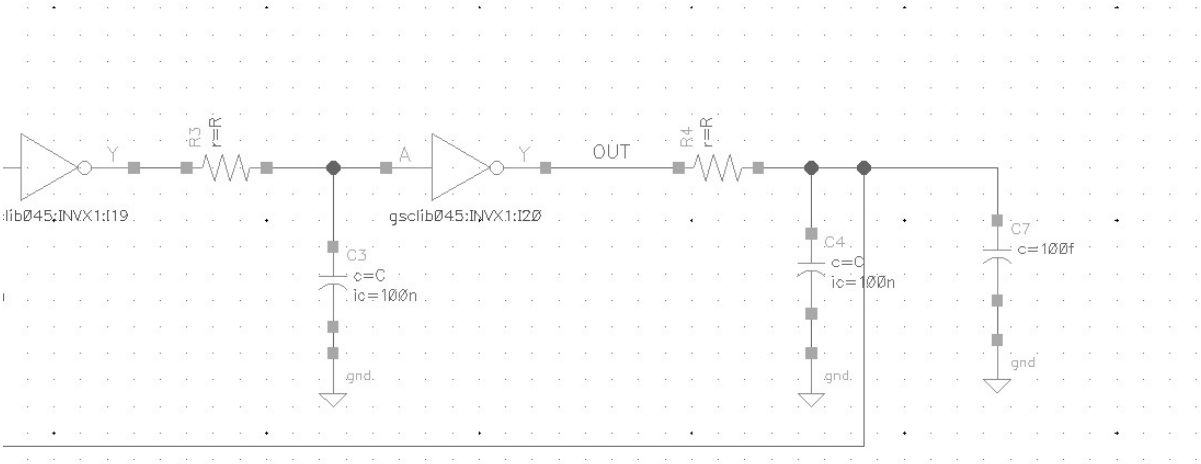


Figure 44 — Close-up of the output node, showing the 100 fF load capacitor C7 in parallel with the last stage.

We run the simulation again and get:

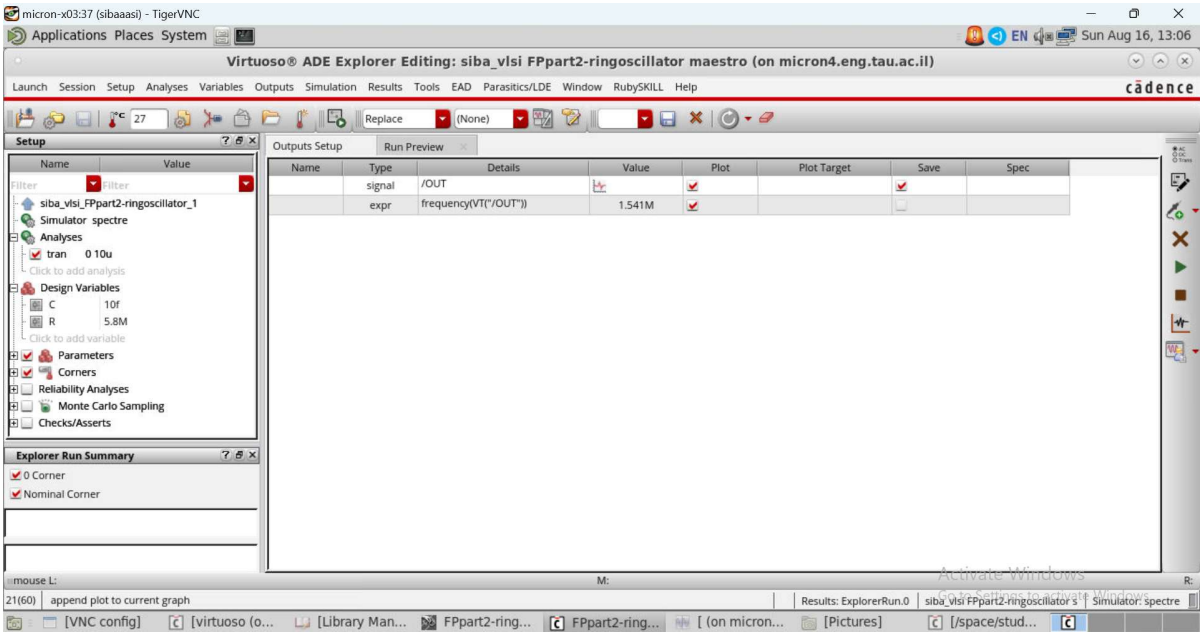


Figure 45 — ADE Explorer run preview for the loaded oscillator. The measured frequency of /OUT has dropped to 1.541 MHz.

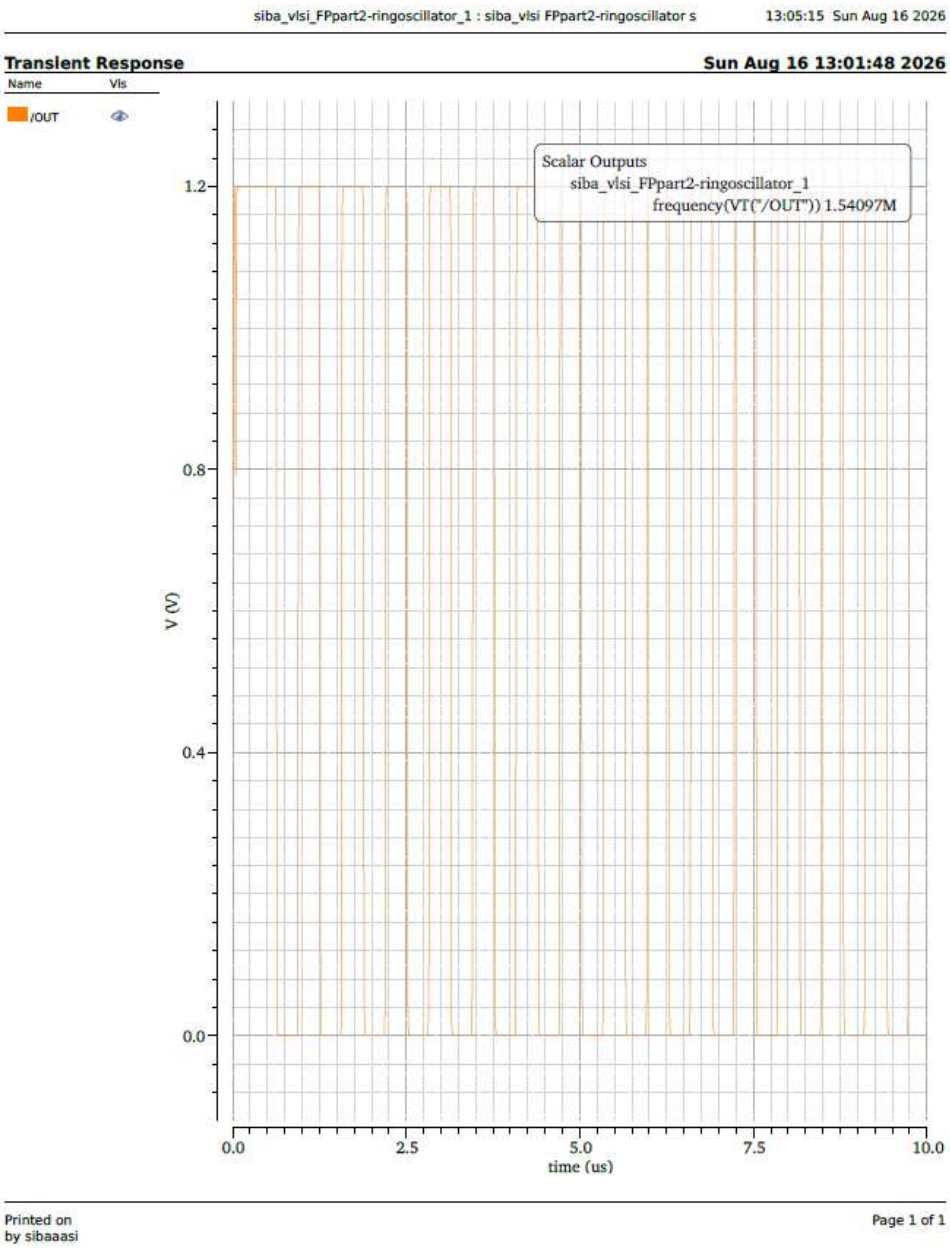


Figure 46 — Transient response with the 100 fF load. frequency(VT("/OUT")) = 1.54097 MHz.

After adding the 100fF load capacitor, the oscillation frequency decreased to: f = 1.54097MHz

This result is expected because the additional load capacitance increases the propagation delay at the output. Since the oscillation frequency is inversely proportional to the total delay, a larger delay leads to a lower frequency.

And this is a close up view:

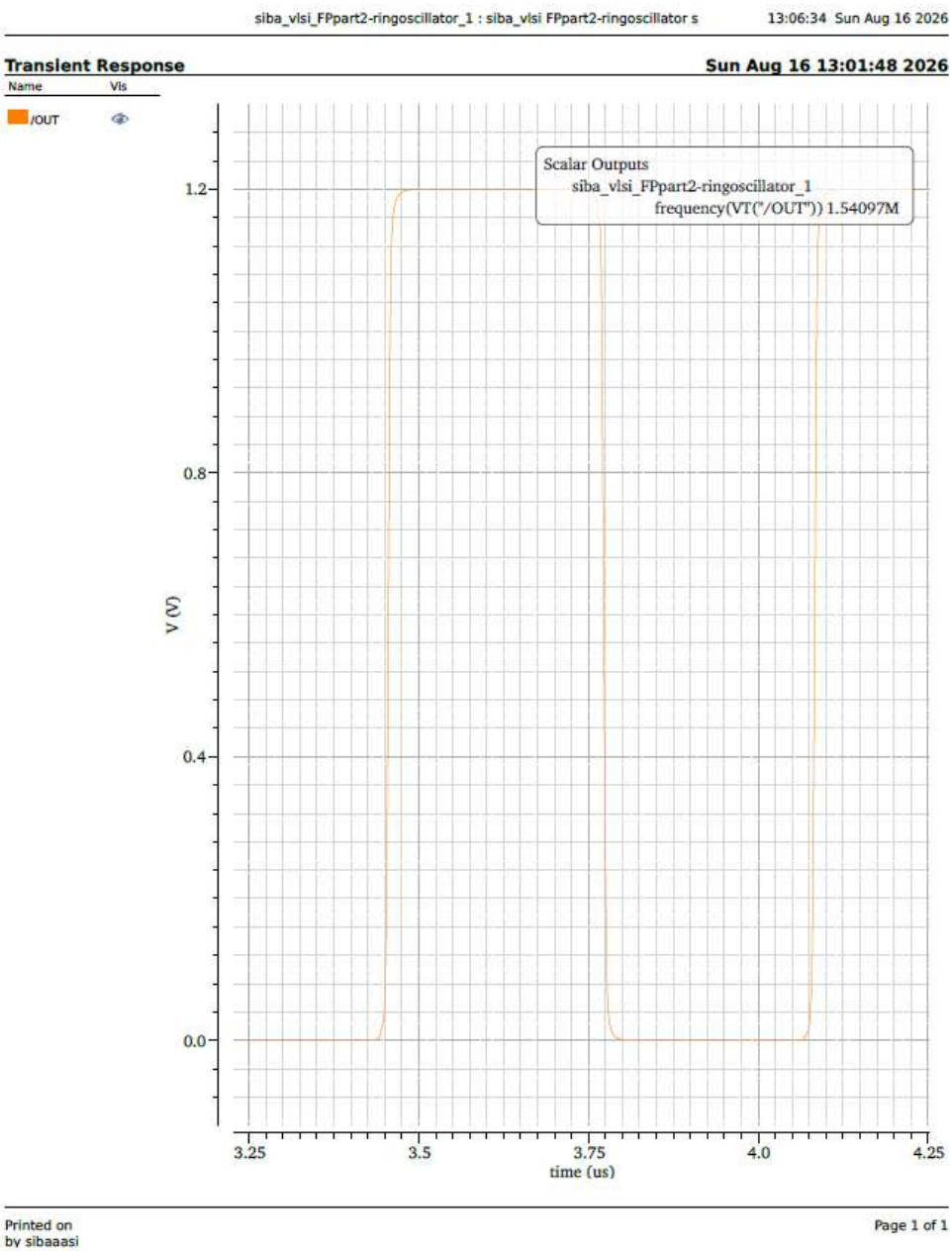


Figure 47 — Close-up of two periods of the loaded oscillator between 3.25 µs and 4.25 µs. The swing is still rail-to-rail but the edges are visibly slower.

The output still exhibits rail-to-rail voltage swings, although the rising and falling edges are slower because of the additional load capacitance. To reduce the effect of this load on the ring oscillator, we added an output buffer that isolates the oscillator from the 100fF capacitor.

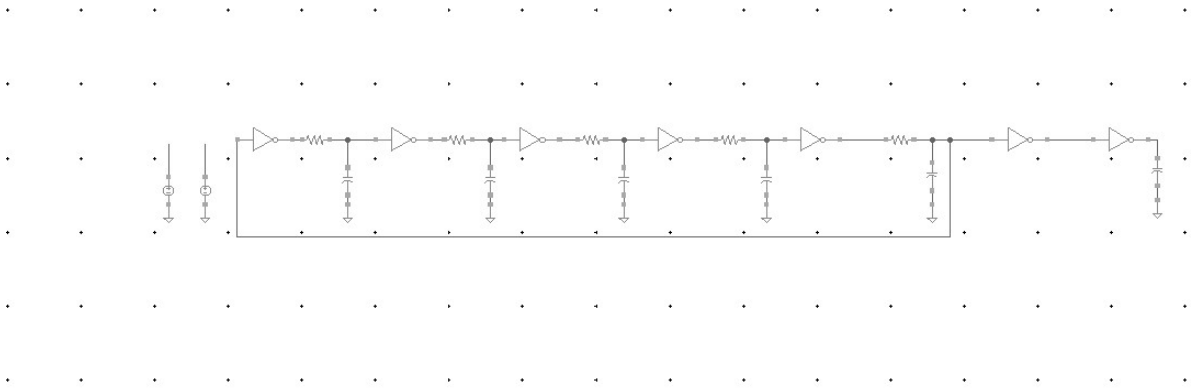


Figure 48 — The ring oscillator with a two-inverter output buffer inserted between the oscillator loop and the 100 fF load.

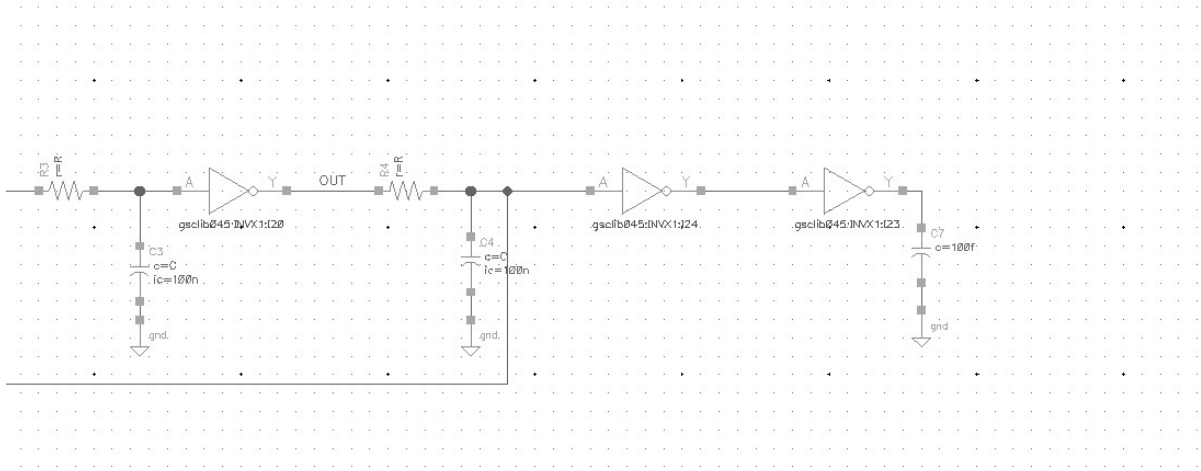


Figure 49 — Close-up of the output buffer — two cascaded INVX1 inverters driving the 100 fF capacitor C7.

We run the simulation again and get:

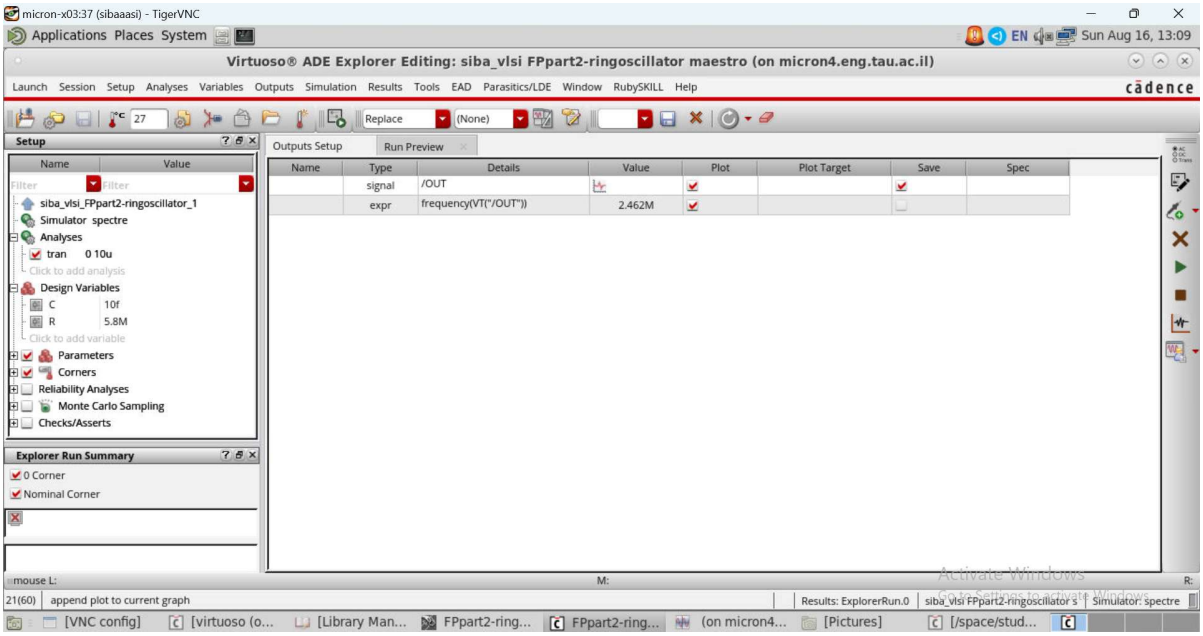


Figure 50 — ADE Explorer run preview with the output buffer. The measured frequency of /OUT is back up to 2.462 MHz.

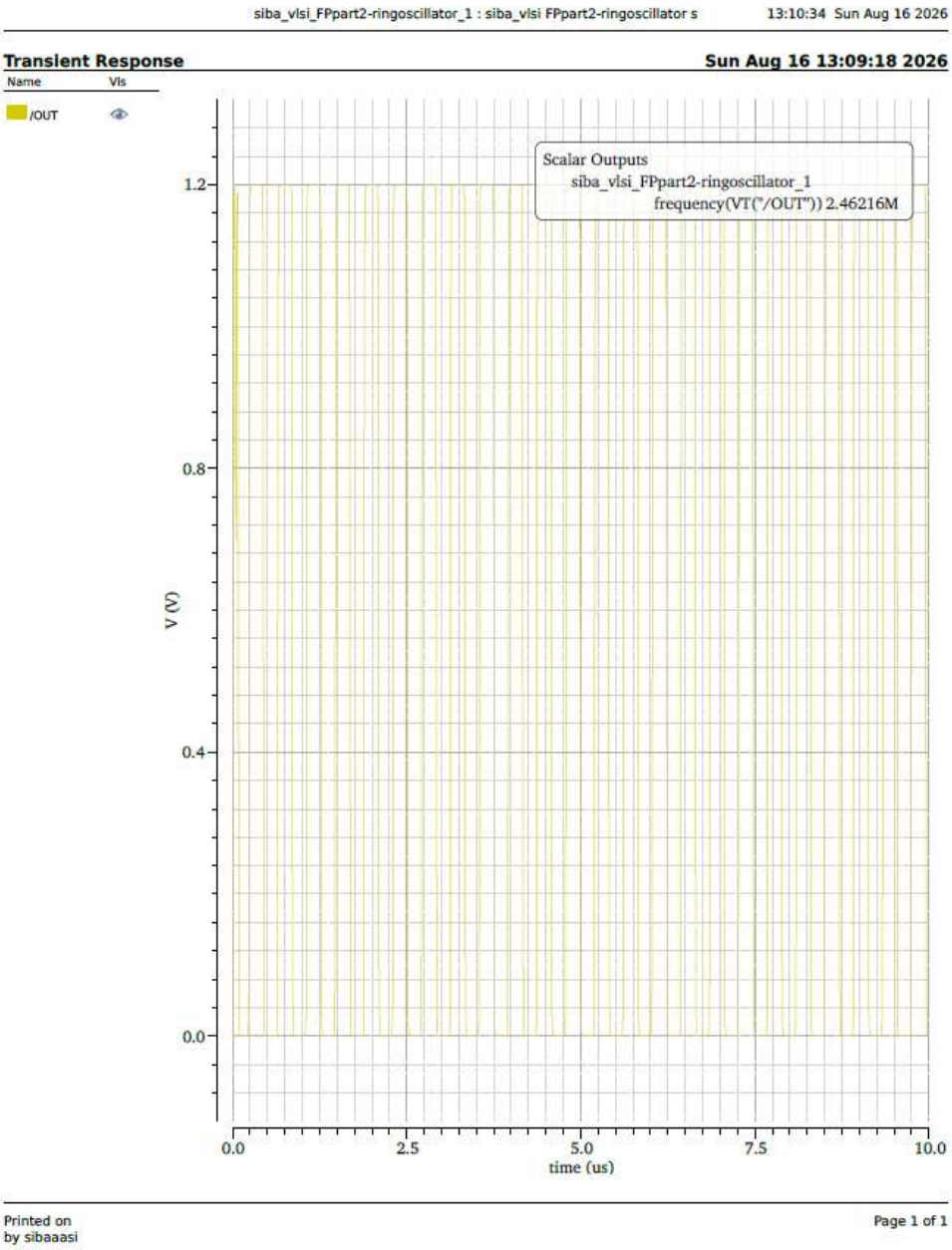


Figure 51 — Transient response of the buffered oscillator. frequency(VT("/OUT")) = 2.46216 MHz.

After adding the output buffer, the oscillation frequency increased to: f = 2.46216MHz

This is a significant improvement compared with the loaded frequency of 1.54097MHz and is close to the original unloaded frequency of 2.498MHz. This result confirms that the buffer effectively isolates the ring oscillator from the 100fF load capacitor.